



(19) **United States**

(12) **Patent Application Publication**

(10) **Pub. No.: US 2025/0284661 A1**

(43) **Pub. Date: Sep. 11, 2025**

(122) **Harres et al.**

(54) **AUTOMATIC FILESYSTEM CLONE GENERATION**

- (71) Applicant: **Oracle International Corporation**,
Redwood Shores, CA (US)
- (72) Inventors: **John Michael Harres**, Thornton, CO (US); **Mark Leroy Shellenbaum**,
Denver, CO (US); **Bhaskar Sarkar**,
Atlanta, GA (US)
- (73) Assignee: **Oracle International Corporation**,
Redwood Shores, CA (US)
- (21) Appl. No.: **19/219,889**
- (22) Filed: **May 27, 2025**

Related U.S. Application Data

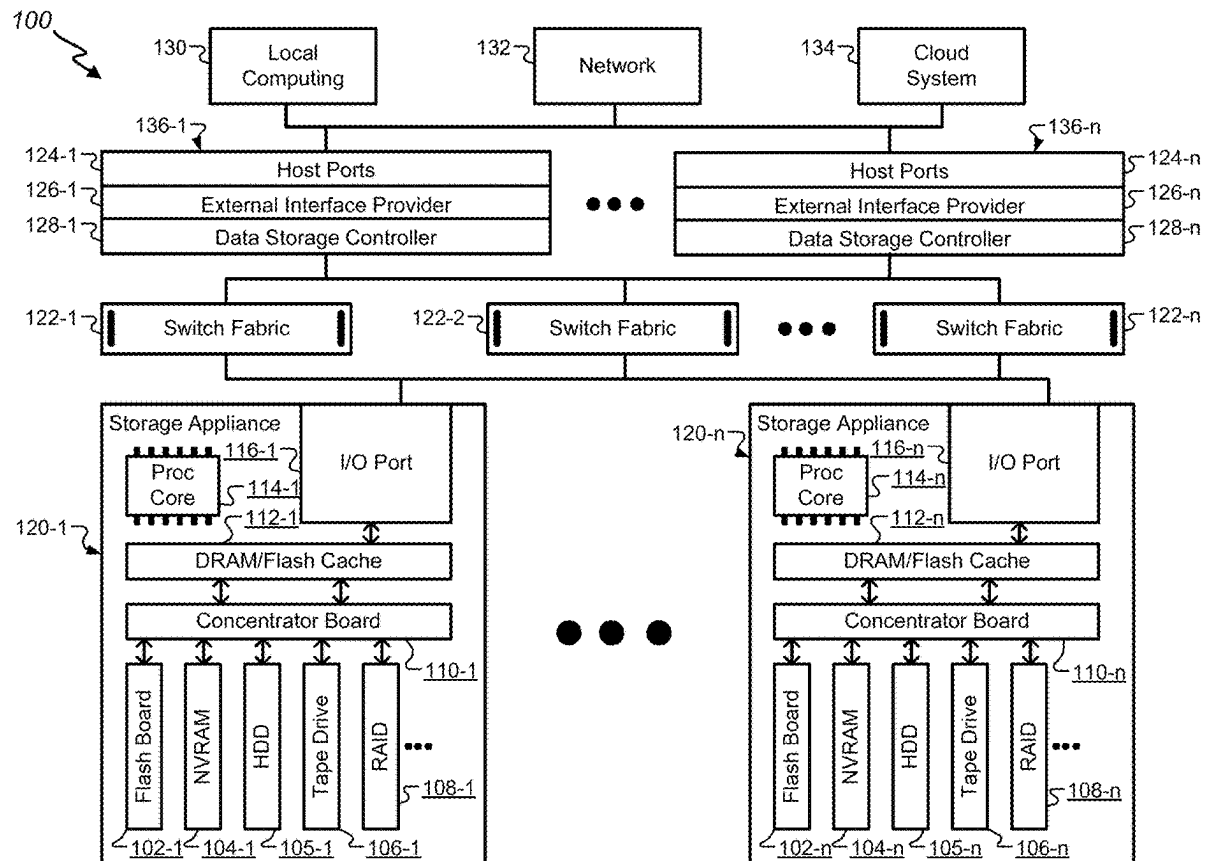
- (63) Continuation of application No. 18/140,474, filed on Apr. 27, 2023, now Pat. No. 12,332,840.

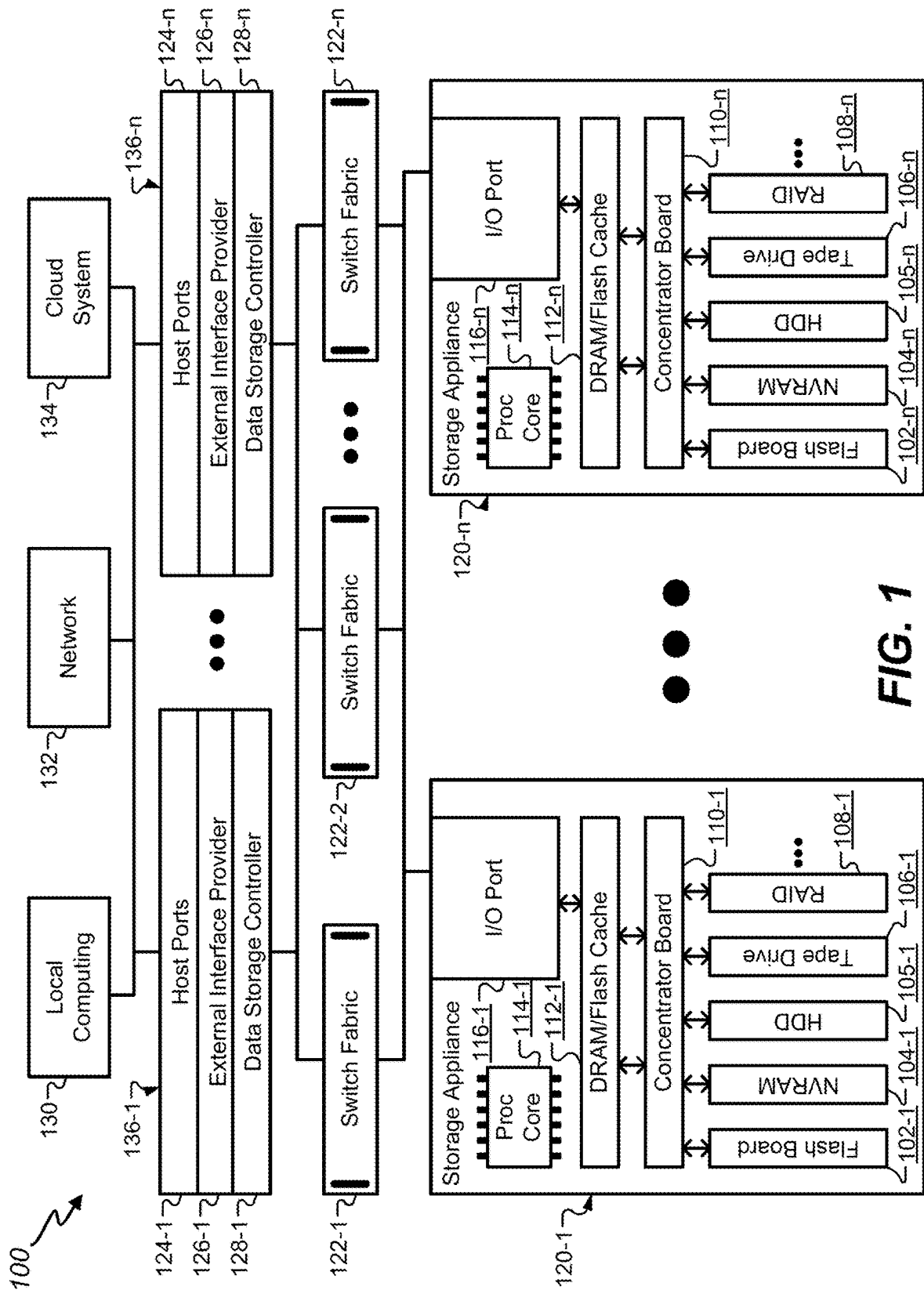
Publication Classification

- (51) **Int. Cl.**
G06F 16/11 (2019.01)
G06F 16/13 (2019.01)
G06F 16/182 (2019.01)
- (52) **U.S. Cl.**
CPC **G06F 16/128** (2019.01); **G06F 16/13** (2019.01); **G06F 16/1844** (2019.01)

(57) **ABSTRACT**

Techniques described herein relate to systems and methods of data storage, and more particularly to automatic file site clone generation. In certain embodiments, a computing device may include receiving a selection indicating that a clone directory is visible in a head node. In response to the selection indicating that the clone directory to be visible in the head node, the computing device may include generating a snapshot of a target file system. Also, the computing device may include duplicating a directory of the target file system into the clone directory. Further, the computing device may include mapping an address of the target file system into the clone directory. The computing device may include linking the clone directory to the head node.





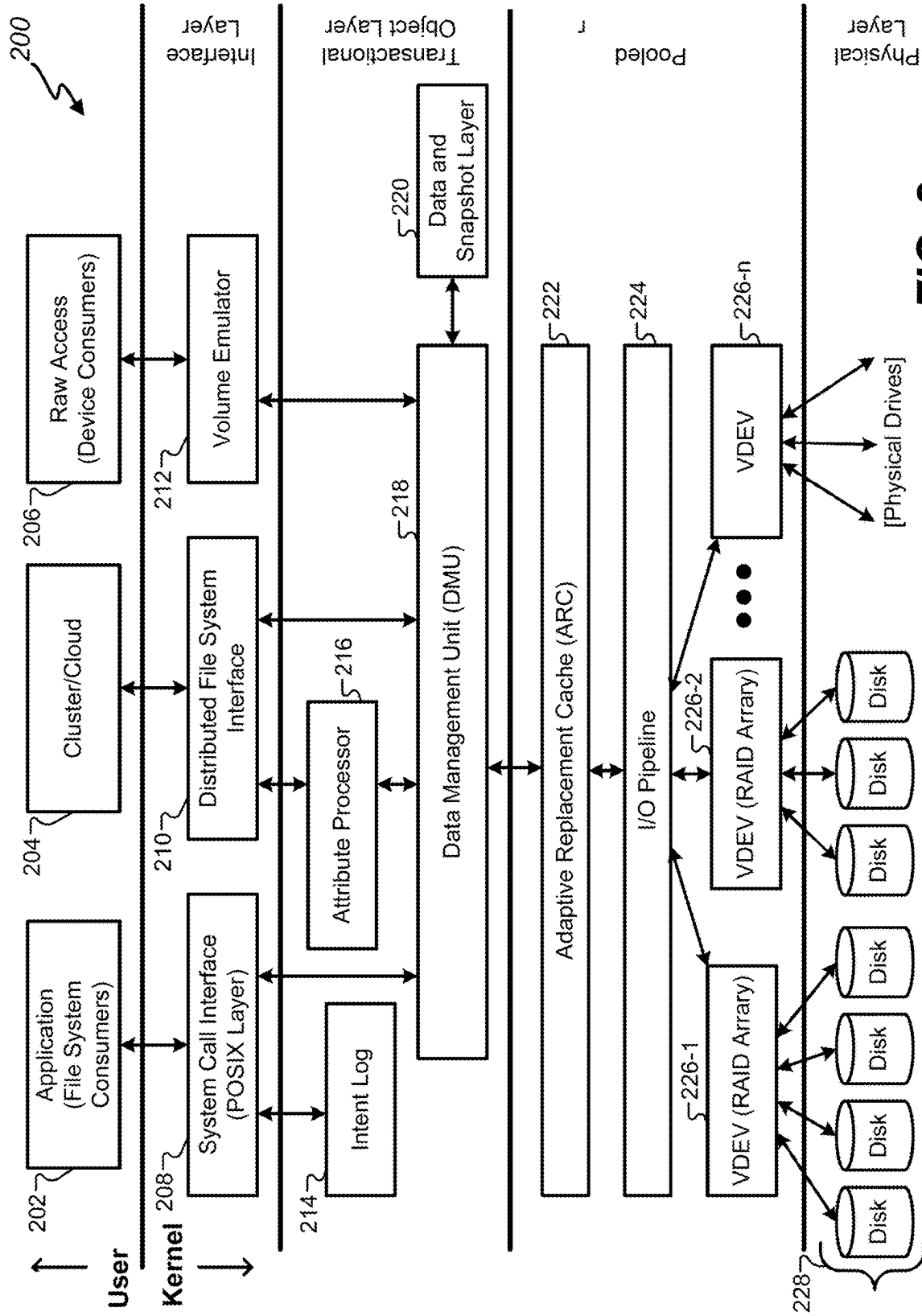


FIG. 2

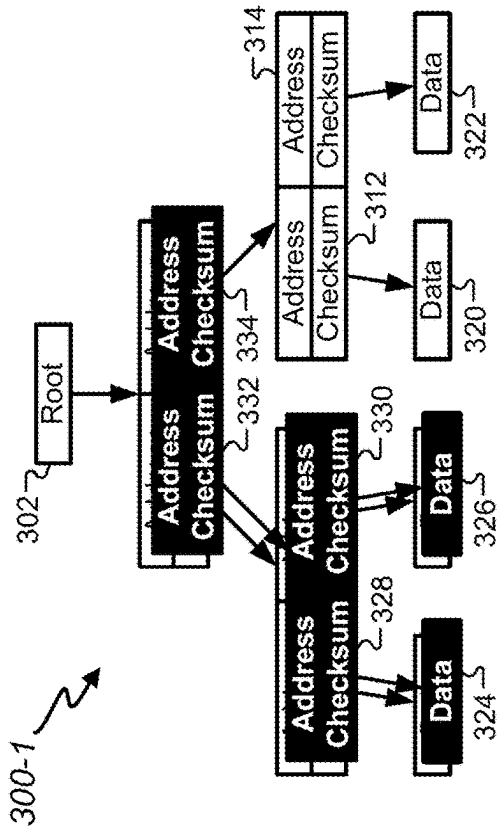


FIG. 3C

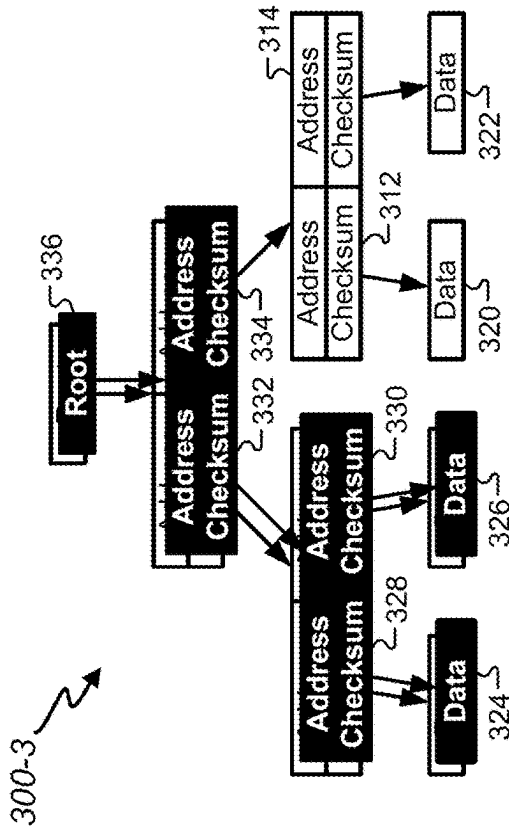


FIG. 3D

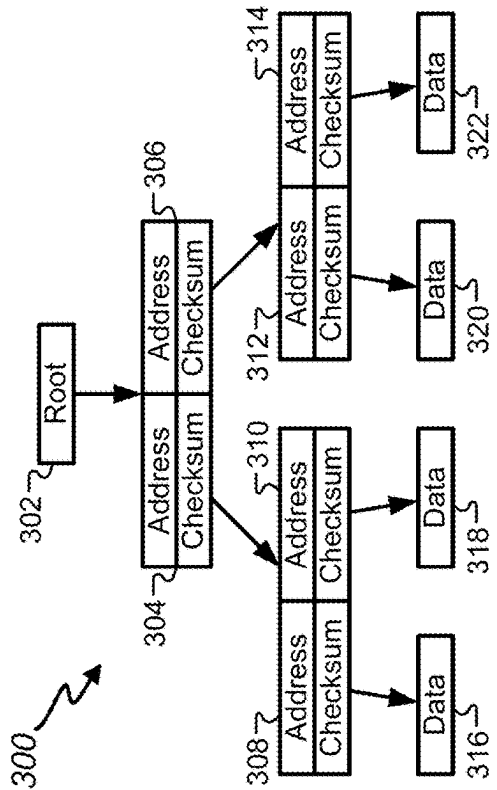


FIG. 3A

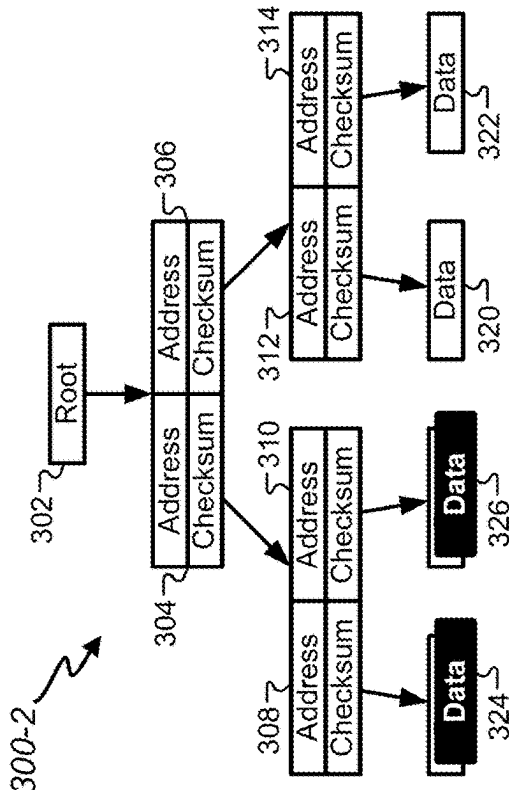


FIG. 3B

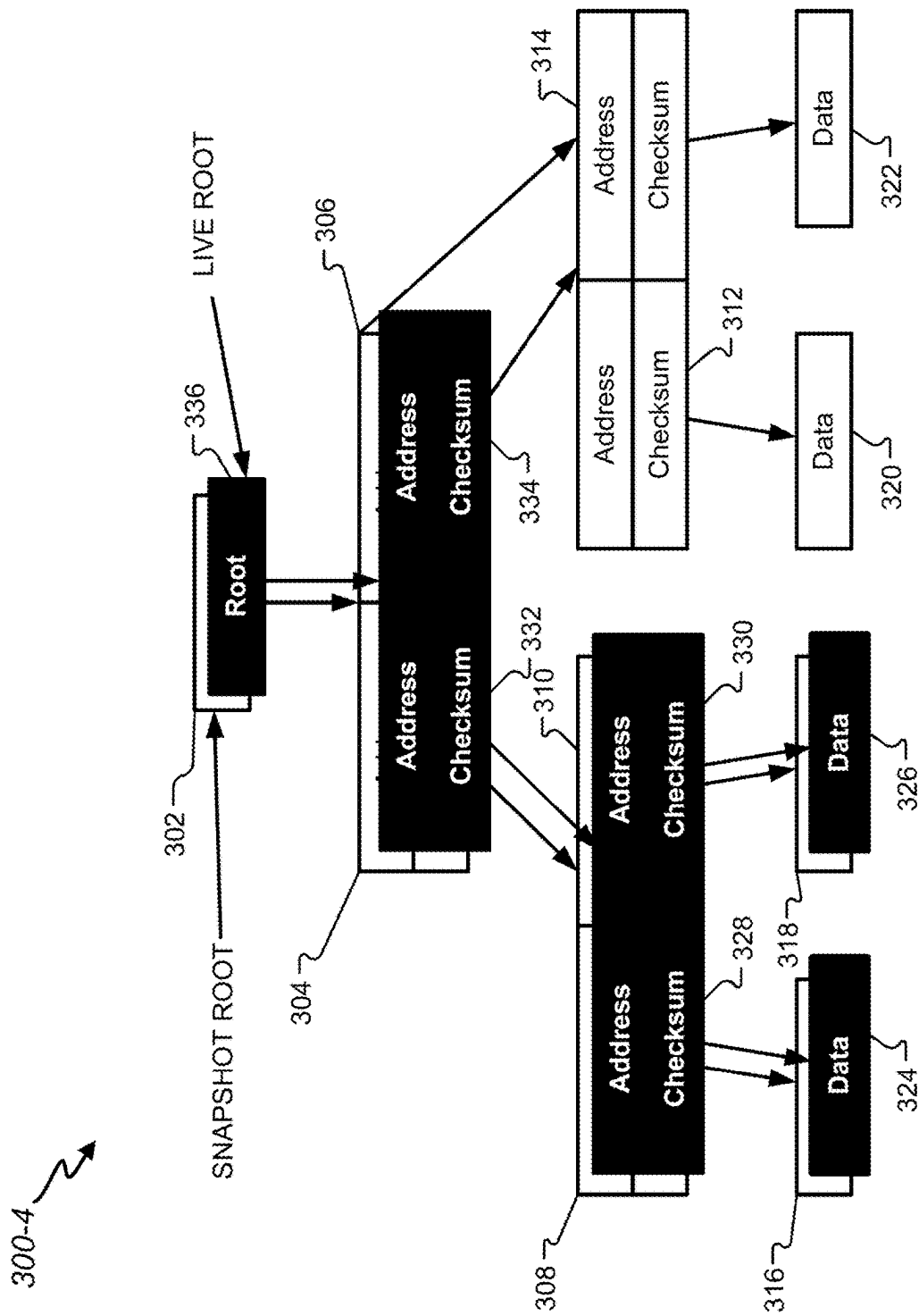


FIG. 3E

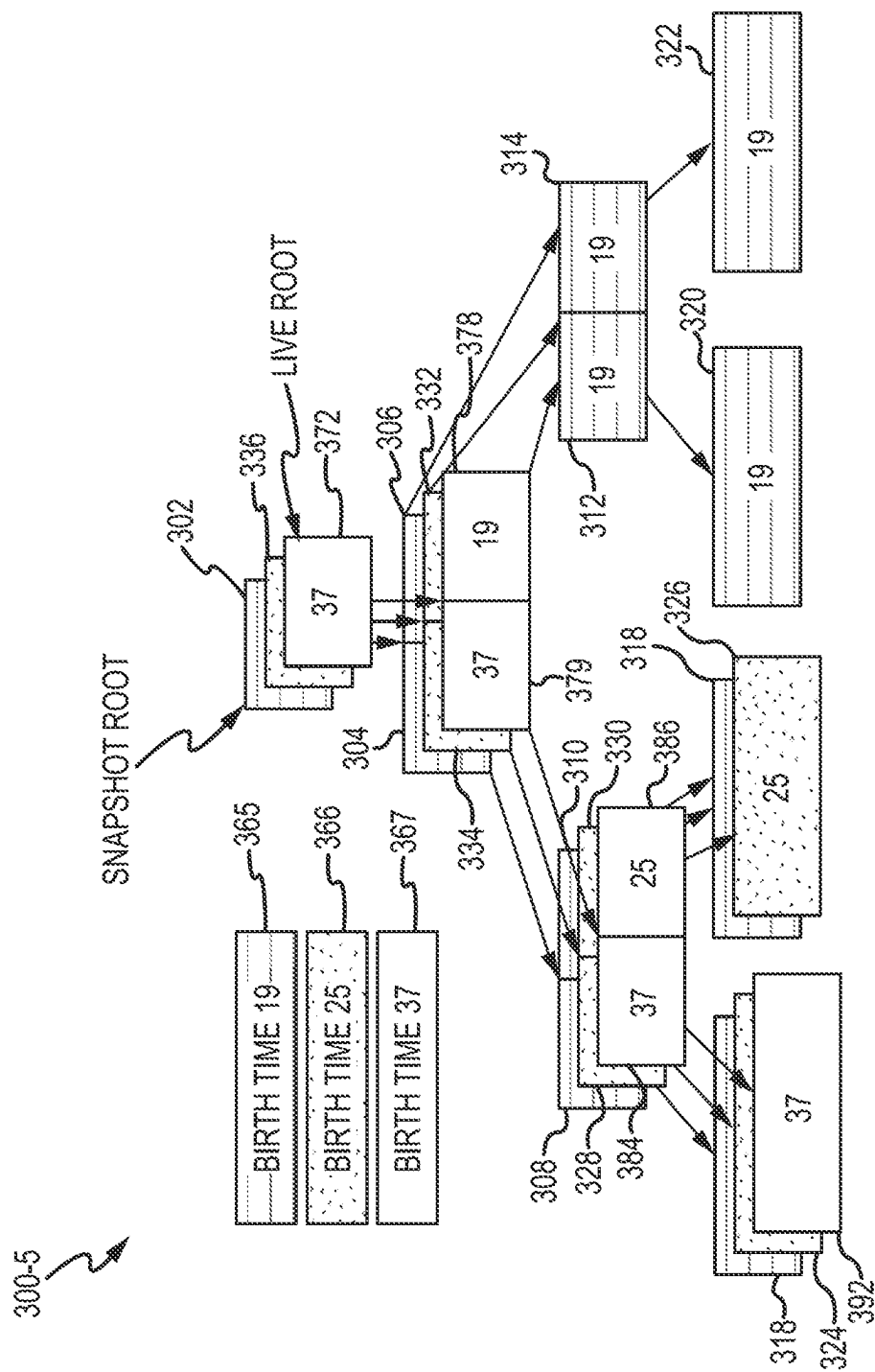


FIG. 3F

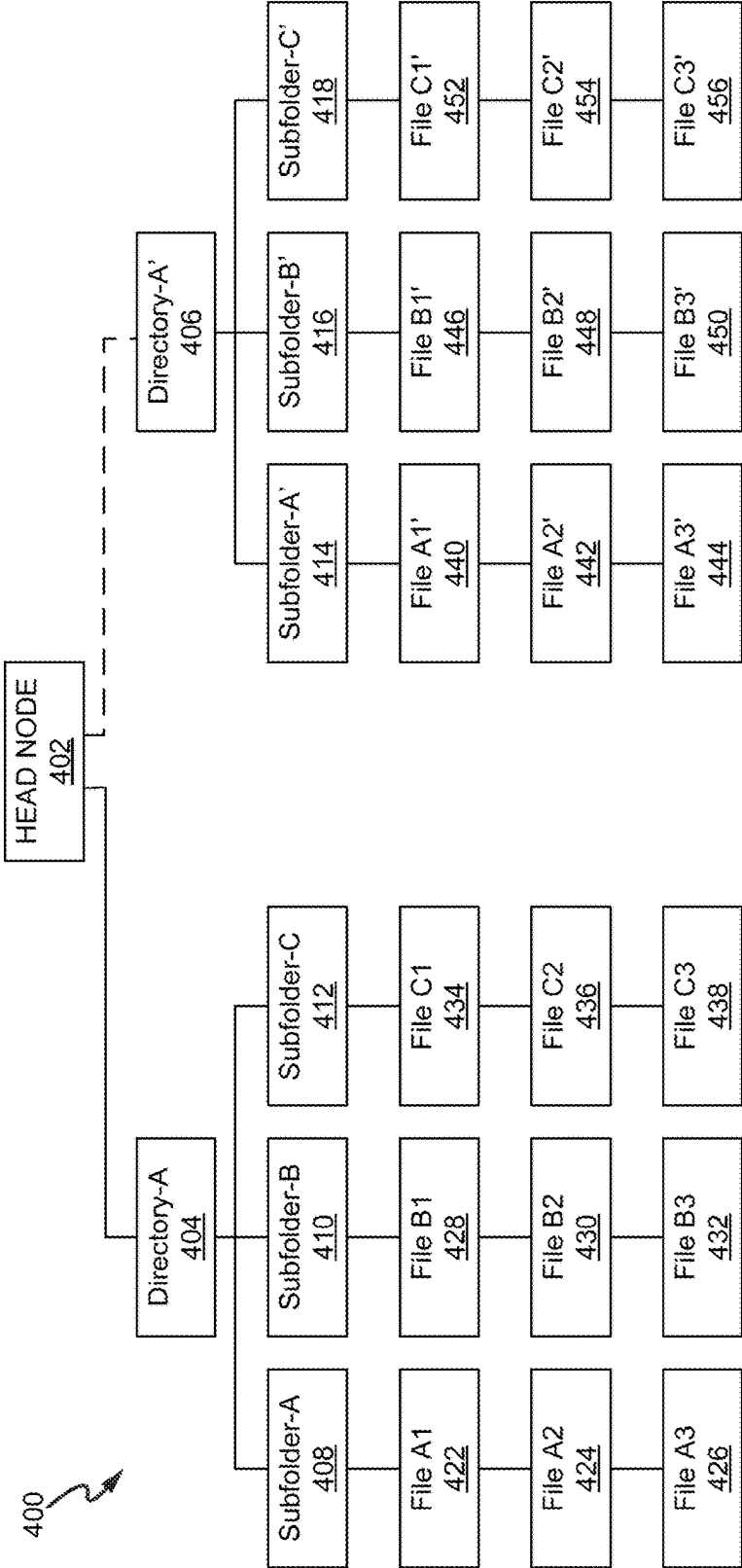
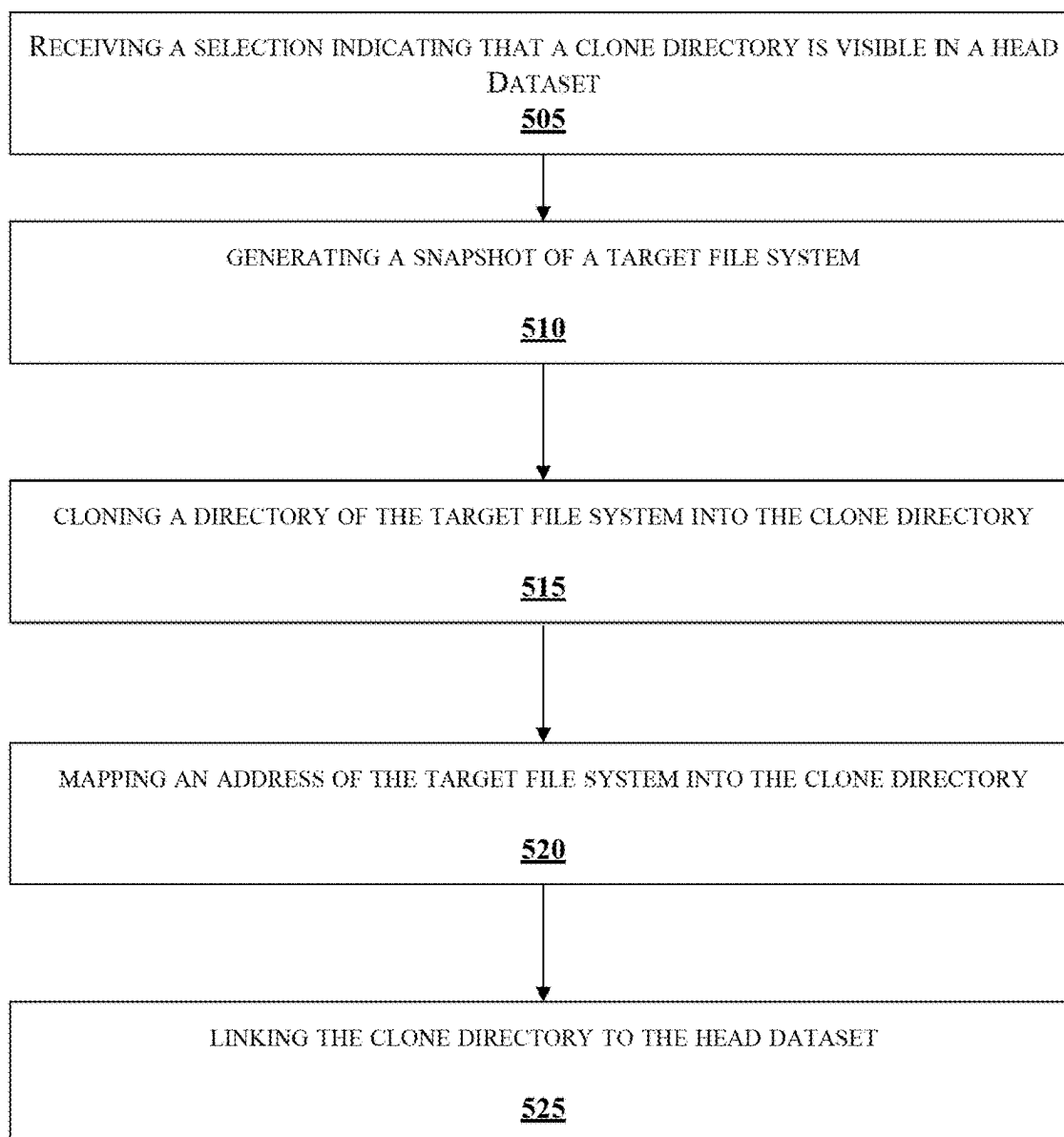


FIG. 4

500

**FIG. 5**

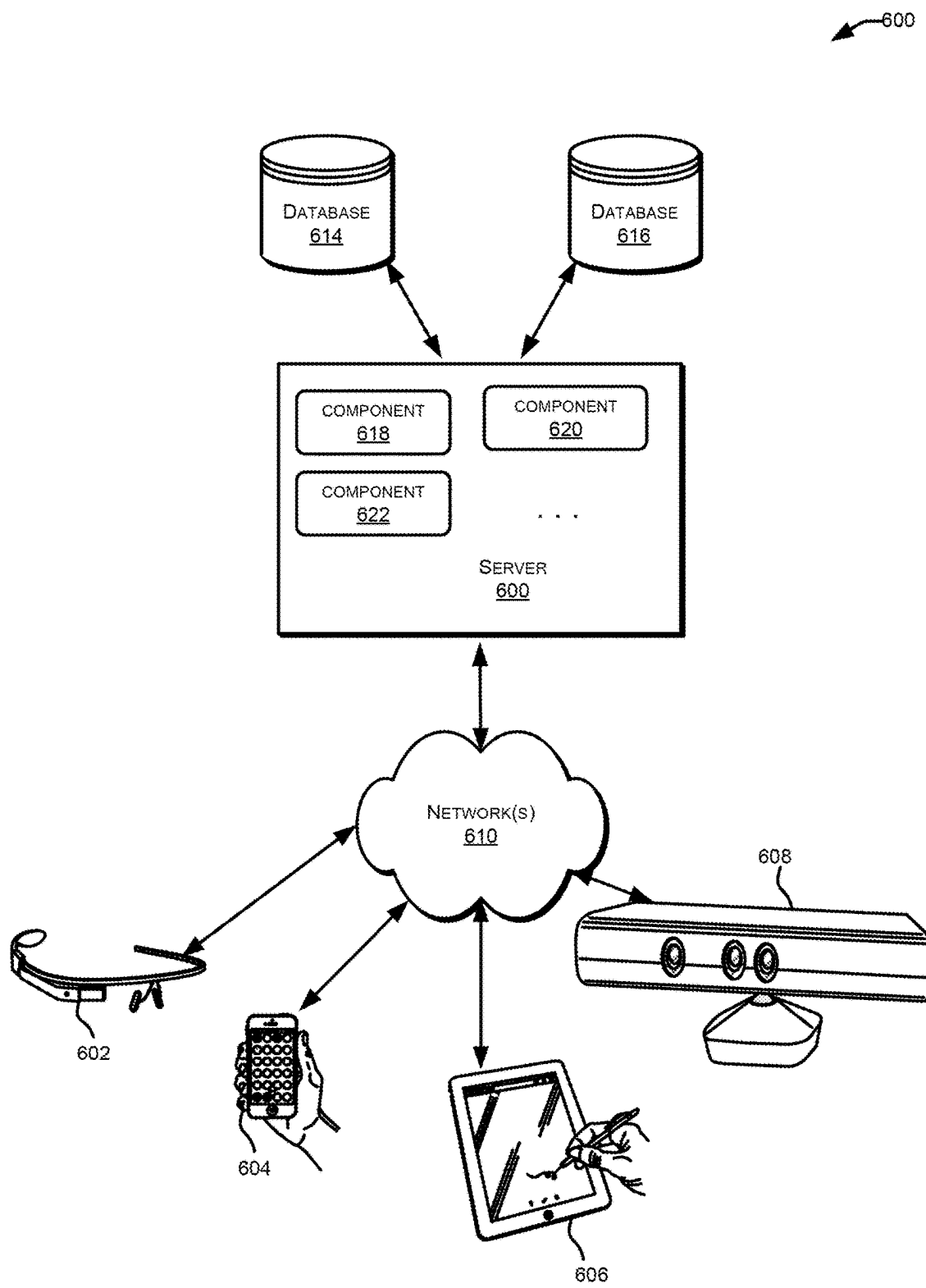


FIG. 6

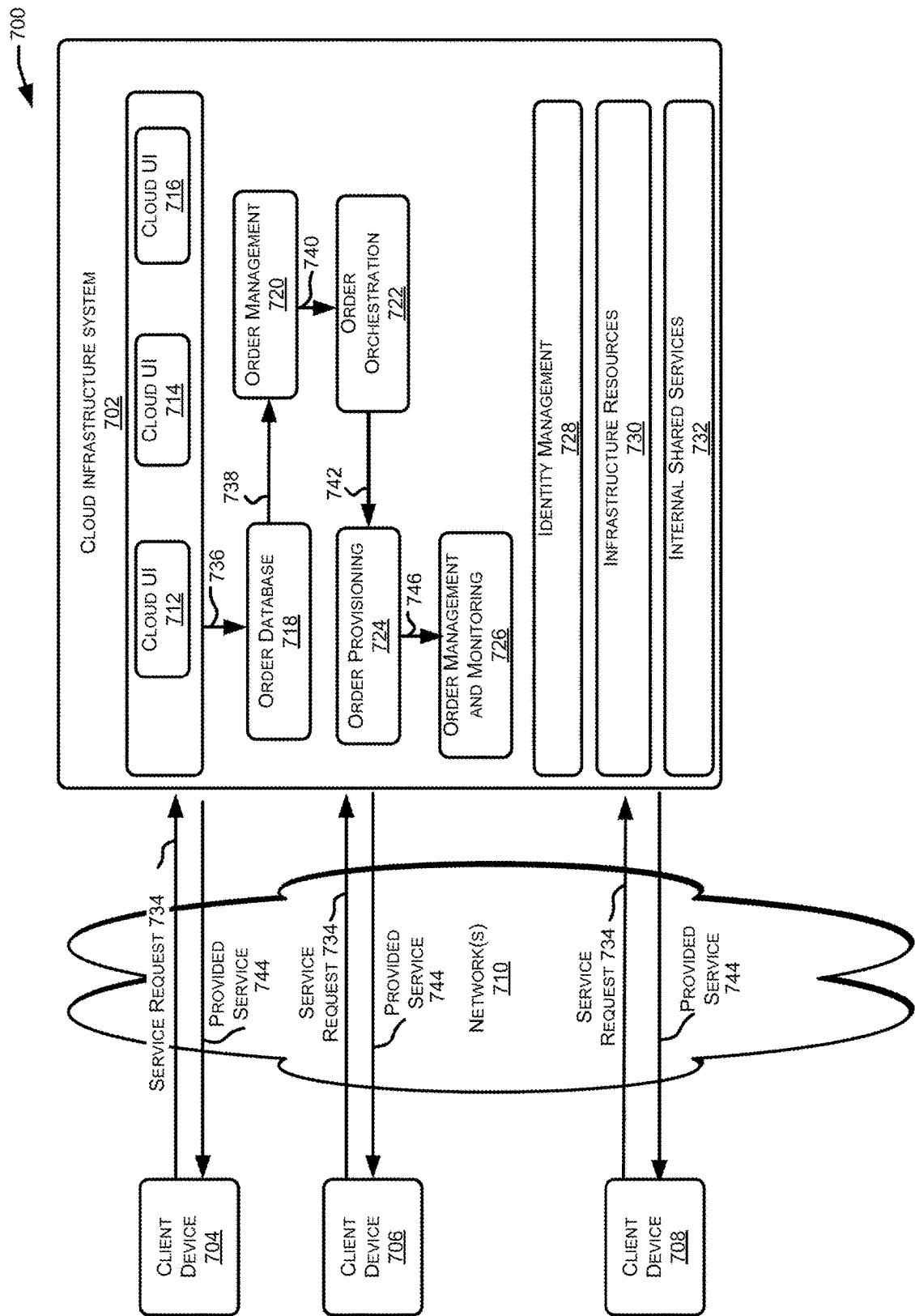


FIG. 7

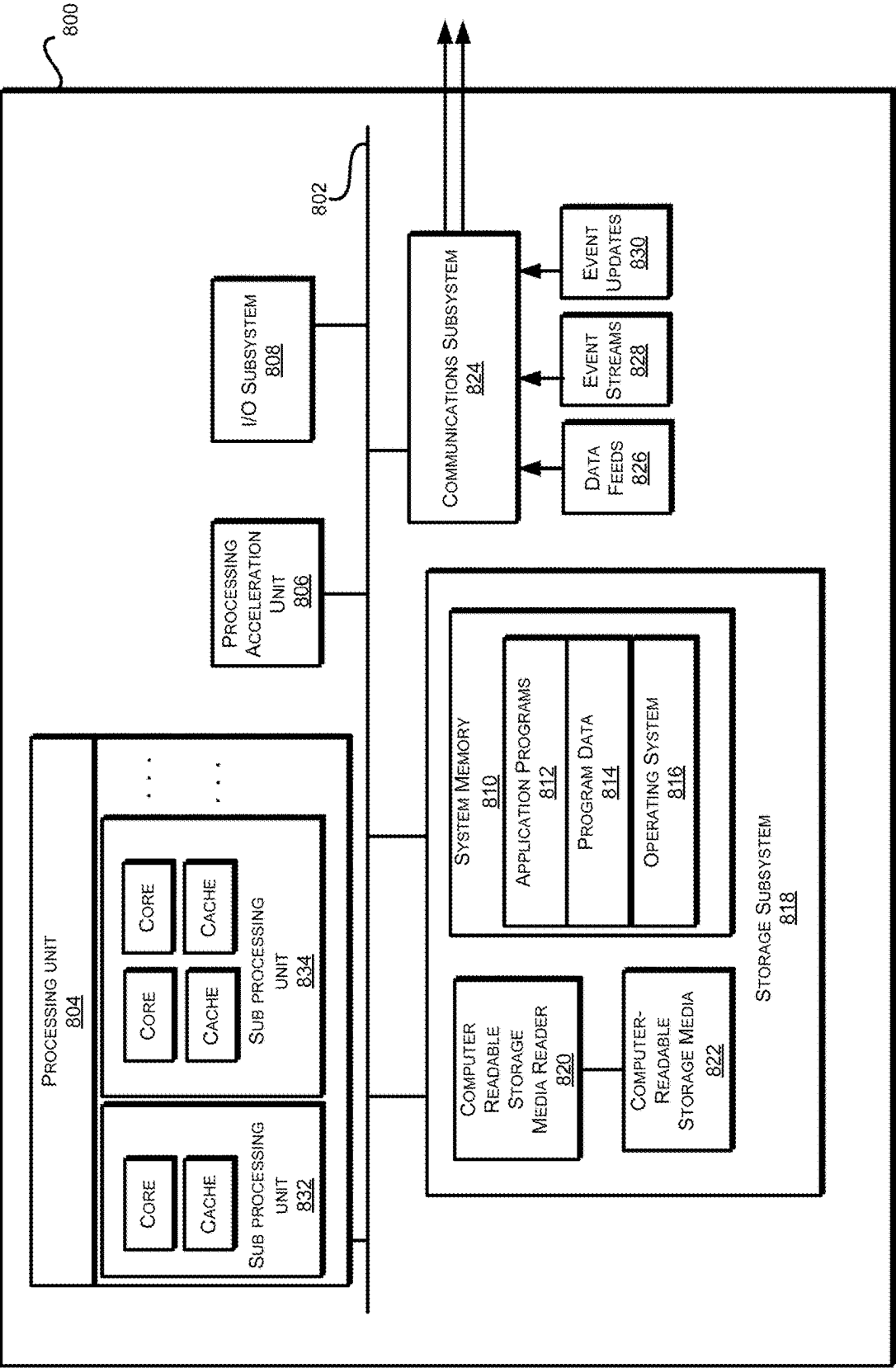


FIG. 8

AUTOMATIC FILESYSTEM CLONE GENERATION

CROSS REFERENCE TO RELATED APPLICATIONS

[0001] This application is a continuation of U.S. patent application Ser. No. 18/140,474, filed on Apr. 27, 2023. The entire disclosure of the aforementioned application is incorporated by reference herein in its entirety for all purposes.

TECHNICAL FIELD OF THE INVENTION

[0002] This disclosure generally relates to systems and methods of data storage, and more particularly to layering file system functionality on an object interface.

BACKGROUND

[0003] The continuous expansion of the Internet, along with the expansion and sophistication of computing networks and systems, has led to the proliferation of content being stored and accessible over the Internet. This, in turn, has driven the need for large and sophisticated data storage systems. As the demand for data storage continues to increase, larger and more sophisticated storage systems are being designed and deployed. Many large-scale data storage systems utilize storage appliances that include arrays of physical storage media. These storage appliances can be capable of storing incredible amounts of data. Moreover, multiple storage appliances may be networked together to form a storage pool, which can further increase the volume of stored data.

[0004] Typically, large storage systems such as these may include a file system for storing and accessing files. In addition to storing system files (operating system files, device driver files, etc.), the file system provides storage and access of user data files. If any of these files (system files and/or user files) contain critical data, then it becomes advantageous to employ a backup storage scheme to ensure that critical data is not lost if a file storage device fails.

[0005] Snapshots are read-only, and the files in a snapshot are not normally able to be edited as a live version. A clone is a new file system that is created based on a snapshot of an existing file system. Snapshots preserve the state of the data of a file system at a particular point in time. If a user takes snapshots of a file system at regular intervals, the user can create clones of the file system as it existed at multiple points in its lifetime.

[0006] Once a clone is generated it can be mounted to allow users to access the files. When a clone is mounted in a cloud computing environment, it means that the clone is made available to the user and can be accessed from a device connected to the network (e.g., the Internet). An administrator may not know which clients have the filesystem mounted nor have the ability to administer every client when new filesystems are made available so as to have them mount them. Currently, an administrator is needed to make a new clone available, and then all clients have to take notice and then act to mount it. The system administrator can take action to create a clone and it is immediately available to all clients with no notification needed, and no action required by the clients. This also allows clients to create and share new clones to all other clients themselves with no administrative access required, no notification for the clients, and no action taken by the other clients.

Therefore, access to the clones would need to be managed using numerous mounting points for all of the clones for a system that are generated. The administrator may not be immediately available to make the connections so the clones can be immediately available to clients.

[0007] Further when a customer duplicates a filesystem to a live copy for use, there are administrative interfaces that can be used to accomplish this requiring security privileges and administrative access.

[0008] Thus, there is a need for systems and methods that address the foregoing problems in order to provide automatic clone generation to alleviate management of the mounting points for the clones. This and other needs are addressed by the present disclosure.

BRIEF SUMMARY

[0009] Certain embodiments of the present disclosure relate generally to systems and methods of data storage, and more particularly to systems and methods for automatic file system clone generation.

[0010] Various techniques (e.g., systems, methods, computer-program products tangibly embodied in a non-transitory machine-readable storage medium, etc.) are described herein for creating a live writable filesystem from a snapshot without actually copying any of the files. These techniques can be done nearly instantly using what is normally only provided by an administrator on administrative interface by a simple filesystem command.

[0011] A system of one or more computers can be configured to perform particular operations or actions by virtue of having software, firmware, hardware, or a combination of them installed on the system that in operation causes or cause the system to perform the actions. One or more computer programs can be configured to perform particular operations or actions by virtue of including instructions that, when executed by data processing apparatus, cause the apparatus to perform the actions.

[0012] In one general aspect, computer implemented method may include receiving a selection indicating that a clone directory is visible in a head node. In response to the selection indicating that the clone directory to be visible in the head node, the computer implemented method may include generating a snapshot of a target file system. The method may also include duplicating a directory of the target file system into the clone directory. The method may further include mapping an address of the target file system into the clone directory. The method may in addition include linking the clone directory to the head node. Other embodiments of this aspect include corresponding computer systems, apparatus, and computer programs recorded on one or more computer storage devices, each configured to perform the actions of the methods.

[0013] Implementations may include one or more of the following features: The computer implemented method may include mirroring security privileges and administrative access from the target file system to the clone directory. In various embodiments, the files in the clone directory are read-write files. The computer implemented method may include receiving a selection of the snapshot of the target file system that is stored in a snapshot directory. In various embodiments, the snapshot is a point-in-time reference of the target file system. The computer implemented method may include copying file system properties of the target file system. In various embodiments, the file system properties

include one or more of compartment, tags, display name, keys, and mount target export information. The computer implemented method may include copying linking metadata from the target file system to the clone directory. Implementations of the described techniques may include hardware, a method or process, or a computer tangible medium.

[0014] In one general aspect, computing system may include one or more processors. Computing system may in addition include a memory in communication with the one or more processors, the memory storing instructions that when executed by the one or more processors perform operations. The operations may include receiving a selection indicating that a clone directory is visible in a head node. In response to the selection indicating that the clone directory to be visible in the head node, the method can include generating a snapshot of a target file system. The operations may include duplicating a directory of the target file system into the clone directory. The operations may include mapping an address of the target file system into the clone directory. The operations may include linking the clone directory to the head node. Other embodiments of this aspect include corresponding computer systems, apparatus, and computer programs recorded on one or more computer storage devices, each configured to perform the actions of the methods.

[0015] Implementations may include one or more of the following features. In various embodiments, the operations may include mirroring security privileges and administrative access from the target file system to the clone directory. In various embodiments, the files in the clone directory are read-write files. In various embodiments, the operations further may include receiving a selection of the snapshot of the target file system that is stored in a snapshot directory. In various embodiments, the snapshot is a point-in-time reference of the target file system. Computer system where the operations further may include copying file system properties of the target file system, where the file system properties include one or more of compartment, tags, display name, keys, and mount target export information. In various embodiments, the operations further may include copying linking metadata from the target file system to the clone directory. Implementations of the described techniques may include hardware, a method or process, or a computer tangible medium.

[0016] In one general aspect, a non-transitory computer readable medium storing instructions to perform operations. The operations can include receiving a selection indicating that a clone directory is visible in a head node. In response to the selection indicating that the clone directory to be visible in the head node, the operations can include generating a snapshot of a target file system. The operations can include duplicating a directory of the target file system into the clone directory. The operations can include mapping an address of the target file system into the clone directory. The operations can include linking the clone directory to the head node. Other embodiments of this aspect include corresponding computer systems, apparatus, and computer programs recorded on one or more computer storage devices, each configured to perform the actions of the methods.

[0017] Implementations may include one or more of the following features. The operations can include mirroring security privileges and administrative access from the target file system to the clone directory. In various embodiments, the files in the clone directory are live read-write files. In

various embodiments, the operations may include receiving a selection of the snapshot of the target file system that is stored in a snapshot directory. In various embodiments, the snapshot is a point-in-time reference of the target file system. In various embodiments, the operations further may include copying file system properties of the target file system. The file system properties can include one or more of compartment, tags, display name, keys, and mount target export information. Implementations of the described techniques may include hardware, a method or process, or a computer tangible medium.

[0018] Further areas of applicability of the present disclosure will become apparent from the detailed description provided hereinafter. It should be understood that the detailed description and specific examples, while indicating various embodiments, are intended for purposes of illustration only and are not intended to necessarily limit the scope of the disclosure.

BRIEF DESCRIPTION OF THE DRAWINGS

[0019] A further understanding of the nature and advantages of embodiments according to the present disclosure may be realized by reference to the remaining portions of the specification in conjunction with the following appended figures.

[0020] FIG. 1 illustrates one example storage network that may be used in accordance with certain embodiments of the present disclosure.

[0021] FIG. 2 illustrates an instance of a file system that may be executed in a storage environment, in accordance with certain embodiments of the present disclosure.

[0022] FIGS. 3A-3F illustrate a copy-on-write process for a file system, in accordance with certain embodiments of the present disclosure.

[0023] FIG. 4 illustrates an exemplary network file system.

[0024] FIG. 5 is a flow chart of an exemplary process for automatic filesystem clone generation.

[0025] FIG. 6 depicts a simplified diagram of a distributed system for implementing certain embodiments in accordance with present disclosure.

[0026] FIG. 7 is a simplified block diagram of one or more components of a system environment by which services provided by one or more components of a system may be offered as cloud services, in accordance with certain embodiments of the present disclosure.

[0027] FIG. 8 illustrates an exemplary computer system, in which various embodiments of the present invention may be implemented.

[0028] In the appended figures, similar components and/or features may have the same reference label. Further, various components of the same type may be distinguished by following the reference label by a dash and a second label that distinguishes among the similar components. If only the first reference label is used in the specification, the description is applicable to any one of the similar components having the same first reference label irrespective of the second reference label.

DETAILED DESCRIPTION

[0029] The ensuing description provides preferred exemplary embodiment(s) only, and is not intended to limit the scope, applicability, or configuration of the disclosure.

Rather, the ensuing description of the preferred exemplary embodiment(s) will provide those skilled in the art with an enabling description for implementing a preferred exemplary embodiment of the disclosure. It should be understood that various changes may be made in the function and arrangement of elements without departing from the spirit and scope of the disclosure as set forth in the appended claims.

[0030] As noted above, cloud-based storage offers elasticity and scale, but presents a number of problems. Cloud object storage offers interfaces that are based on getting and putting whole objects. Cloud object storage provides a limited ability to search, and typically has high latency. The limited cloud-based interfaces do not align with needs of local file system applications. Converting legacy applications to use an object interface would be expensive and may not be practical or even possible. Hence, solutions are needed so that is not necessary to change file system applications to directly access cloud object storage because it promises to be complex and expensive.

[0031] The solutions should allow native application interfaces to be preserved without introducing various types of adaptation layers to map the data of a local storage system to object storage in the cloud. Accordingly, certain embodiments according to the present disclosure may layer file system functionality on cloud object interfaces to provide cloud-based storage while allowing for functionality expected from a legacy applications. For example, legacy applications, which are not cloud-based, may access data primarily as files and may be configured for POSIX interfaces and semantics. From the legacy application perspective, being able to modify content of a file without rewriting the file is expected. Likewise, being able to organize data in name hierarchies is expected.

[0032] To accommodate such expectations, certain embodiments may layer POSIX interfaces and semantics on cloud-based storage, while providing access to data in a manner that, from a user perspective, is consistent with file-based access with data organization in name hierarchies. Further, certain embodiments may provide for memory mapping of data so that memory map changes are reflected in persistent storage while ensuring consistency between memory map changes and writes. By transforming a ZFS file system disk-based storage into ZFS cloud-based storage, the ZFS file system gains the elastic nature of cloud storage. By mapping “disk blocks” to cloud objects, the storage requirements for the ZFS file system are only the “blocks” actually in use. The system may always be thinly provisioned with no risk of running out of backing storage. Conversely, the cloud storage gains ZFS file system semantics and services. Full POSIX semantics may be provided to cloud clients, as well as any additional data services (such as compression, encryption, snapshots, etc.) provided by the ZFS file system.

[0033] Certain embodiments may provide the ability to migrate data to-and-from the cloud and may provide for local data to co-exist with data in the cloud by way of a hybrid cloud storage system, which provides for storage elasticity and scale while layering ZFS file system functionality on the cloud storage. By extending the ZFS file system to allow storing of objects in cloud object stores, a bridge may be provided to traditional object storage while preserving ZFS file system functionality in addition to all ZFS file system data services. The bridging of the gap between

traditional local file systems and the ability to store data in various clouds object stores redounds to significant performance improvements.

[0034] Furthermore, embodiments of the present invention enable the use of traditional ZFS data services in conjunction with the hybrid cloud storage. As an example, compression, encryption, deduplication, snapshots, and clones are each available in certain embodiments of the present invention and are described in short detail immediately below. In the present invention users can continue to use all of the data services provided by the ZFS file system seamlessly when extending storage to the cloud. For instance, the Oracle Key Manager or equivalent manages the key local to a user allowing end-to-end secure encryption with locally managed keys when storing to the cloud. The same commands used to compress, encrypt, deduplicate, take snapshots, and make clones on disk storage are used for storage to the cloud. As a result, users continue to benefit from the efficiencies and the security provided by ZFS compression, encryption, deduplication, snapshots, and clones.

[0035] Compression is typically turned on because it reduces the resources required to store and transmit data. Computational resources are consumed in the compression process and, usually, in the reversal of the process (decompression). Data compression is subject to a space-time complexity trade-off. For instance, a compression scheme may require intensive processing decompression fast enough to be consumed as it is being decompressed. The design of data compression schemes involves trade-offs among various factors, including the degree of compression and the computational resources required to compress and decompress the data.

[0036] ZFS encryption enables an end-to-end secure data block system with locally saved encryption keys providing an additional layer of security. ZFS encryption does not of itself prevent the data blocks from being misappropriated but denies the message content to the interceptor. In an encryption scheme, the intended data block, is encrypted using an encryption algorithm, generating ciphertext that can only be read if decrypted. For technical reasons, an encryption scheme usually uses a pseudo-random encryption key generated by an algorithm. It is in principle possible to decrypt the message without possessing the key, but, for a well-designed encryption scheme, large computational resources and skill are required. ZFS Data blocks are encrypted using AES (Advanced Encryption Standard) with key lengths of **128**, **192**, and **256**.

[0037] Data block duplication is a specialized data compression technique for eliminating duplicate copies of repeating data blocks. Data block deduplication is used to improve storage utilization and can also be applied to network data transfers to reduce the number of data blocks that must be sent to store to memory. In the deduplication process, unique data blocks are identified and stored during a process of analysis. As the analysis continues, other data blocks are compared to the stored copy and whenever a match occurs, the redundant data block is replaced with a small reference that points to the stored data block. Given that the same data block pattern may occur dozens, hundreds, or even thousands of times the number of data blocks that must be stored or transferred is reduced substantially using deduplication.

[0038] Snapshots for ZFS storage to the cloud object store are created seamlessly in the ZFS system. Snapshots freeze certain data and metadata blocks so that they may not be written over in case a backup to the snapshot is needed. A tree hierarchy can have many snapshots, and each snapshot will be saved until deleted. Snapshots can be stored locally or in the cloud object store. And snapshots are “free” in the ZFS system as they don’t require any extra storage capability than creating the root block the snapshot points to. The root block and all subsequent blocks from the root block are not available for copy on write operation when accessed from the snapshot reference to the root block. At the next progression after the snapshot is taken a new root block becomes the active root block.

[0039] Clones are created from snapshots and, unlike snapshot, blocks accessed using the clone reference to the root block are available for copy on write operation. Clones allow development and troubleshooting on a system without corrupting the active root block and tree. Clones are linked to snapshots and snapshots cannot be deleted if a clone linking to a snapshot block persists. Clones, in some cases, can be promoted to the active hierarchical tree.

[0040] Various techniques (e.g., systems, methods, computer-program products tangibly embodied in a non-transitory machine-readable storage medium, etc.) are described herein for creating a live writable filesystem from a snapshot without actually copying any of the files. These techniques can be done nearly instantly using what is normally only provided by an administrator on administrative interface by a simple filesystem command.

[0041] The computer implemented method can include receiving a selection indicating that a clone directory is visible in a head node. In response to the selection indicating that the clone directory to be visible in the head node, the method can include generating a snapshot of a target file system. The method can include duplicating a directory of the target file system into the clone directory. The method can include mapping an address of the target file system into the clone directory. The method can include linking the clone directory to the head node.

[0042] Various embodiments will now be discussed in greater detail with reference to the accompanying figures, beginning with FIG. 1.

[0043] FIG. 1 illustrates one example storage network 100 that may be used to implement certain embodiments according to the present disclosure. The selection and/or arrangement of hardware devices depicted in FIG. 1 are shown only by way of example and are not meant to be limiting. FIG. 1 provides a plurality of storage appliances 120 connected through one or more switch circuits 122. The switch circuits 122 may connect the plurality of storage appliances 120 to a plurality of I/O servers 136, which in turn may provide access to the plurality of storage appliances 120 for client devices, such as local computer systems 130, computer systems available over a network 132, and/or cloud computing systems 134.

[0044] Each I/O server 136 may execute multiple independent file system instances, each of which may be responsible for the management of a portion of the overall storage capacity. As will be described in greater detail below, these file system instances may include the Oracle ZFS file system. The I/O servers 136 may comprise blade and/or standalone servers that include host ports 124 to communicate with the client devices by receiving read and/or write

data access requests. The host ports 124 may communicate with an external interface provider 126 that identifies a correct data storage controller 128 to service each I/O request. The data storage controllers 128 can each exclusively manage a portion of data content in one or more of the storage appliances 120 described below. Thus, each data storage controller 128 can access a logical portion of the storage pool and satisfy data requests received from the external interface providers 126 by accessing their own data content. Redirection through the data storage controllers 128 may include redirection of each I/O request from the host ports 124 to a file system instance (e.g., a ZFS instance) executing on the I/O servers 136 and responsible for the blocks requested. For example, this may include a redirection from a host port 124-1 on one I/O server 136-1 to a ZFS instance on another I/O server 136-n. This redirection may allow any part of the available storage capacity to be reached from any host port 124. The ZFS instance may then issue the necessary direct I/O transactions to any storage device in the storage pool to complete the request. Acknowledgements and/or data may then be forwarded back to the client device through the originating host port 124.

[0045] A low-latency, memory-mapped network may tie together the host ports 124, any file system instances, and the storage appliances 120. This network may be implemented using one or more switch circuits 122, such as Oracle’s Sun Data Center InfiniBand Switch 36 to provide a scalable, high-performance cluster. A bus protocol, such as the PCI Express bus, may route signals within the storage network. The I/O servers 136 and the storage appliances 120 may communicate as peers. The redirection traffic and ZFS memory traffic may both use the same switch fabric.

[0046] In various embodiments, many different configurations of the storage appliances 120 may be used in the network of FIG. 1. In some embodiments, the Oracle ZFS Storage Appliance series may be used. The ZFS Storage Appliance provides storage based on the Oracle Solaris kernel with Oracle’s ZFS file system (“ZFS”) described below. The processing core 114 handles any operations required to implement any selected data protection (e.g., mirroring, RAID-Z, etc.), data reduction (e.g., inline compression, duplication, etc.), and any other implemented data services (e.g., remote replication, etc.). In some embodiments, the processing core may comprise an 8x15 core of 2.8 GHz Intel® Xeon® processors. The processing core also handles the caching of stored data in both DRAM and Flash 112. In some embodiments, the DRAM/Flash cache may comprise a 3 TB DRAM cache.

[0047] In some configurations, the storage appliances 120 may comprise an I/O port 116 to receive I/O requests from the data storage controllers 128. Each of the storage appliances 120 may include an integral rack-mounted unit with its own internally redundant power supply and cooling system. A concentrator board 110 or other similar hardware device may be used to interconnect a plurality of storage devices. Active components such as memory boards, concentrator boards 110, power supplies, and cooling devices may be hot swappable. For example, the storage appliance 120 may include flash memory 102, nonvolatile RAM (e.g., NVRAM) 104, various configurations of hard disk drives 105, tape drives, RAID arrays 108 of disk drives, and so forth. These storage units may be designed for high availability with hot swapping and internal redundancy of memory cards, power, cooling, and interconnect. In some

embodiments the RAM may be made non-volatile by backing it up to dedicated Flash on loss of power. The mix of Flash and NVRAM cards may be configurable, and both may use the same connector and board profile.

[0048] Although not shown explicitly, each of the I/O servers **136** may execute a global management process, or data storage system manager, that may supervise the operation of the storage system in a pseudo-static, “low touch” approach, intervening when capacity must be reallocated between ZFS instances, for global Flash wear leveling, for configuration changes, and/or for failure recovery. The “divide and conquer” strategy of dividing the capacity among individual ZFS instances may enable a high degree of scalability of performance, connectivity, and capacity. Additional performance may be achieved by horizontally adding more I/O servers **136**, and then assigning less capacity per ZFS instance and/or fewer ZFS instances per I/O server **136**. Performance may also be scaled vertically by using faster servers. Additional host ports may be added by filling available slots in the I/O servers **136** and then adding additional servers. Additional capacity may also be achieved by adding additional storage appliances **120** and allocating the new capacity to new or existing ZFS instances.

[0049] FIG. 2 illustrates an instance of an example network file system **200** that may be executed in a storage environment, including the storage environment of FIG. 1, in accordance with certain embodiments of the present disclosure. For example, the file system **200** may include the Oracle ZFS file system (“ZFS”), which provides very large capacity (128-bit), data integrity, an always-consistent, on-disk format, self-optimizing performance, and real-time remote replication. Among other ways, ZFS departs from traditional file systems at least by eliminating the need for a separate volume manager. Instead, a ZFS file system shares a common storage pool of storage devices and acts as both the volume manager and the file system. Therefore, ZFS has complete knowledge of both the physical disks and volumes (including their condition, status, and logical arrangement into volumes, along with all the files stored on them). Devices can be added or removed from the pool as file system capacity requirements change over time to dynamically grow and shrink as needed without needing to repartition the underlying storage pool.

[0050] In certain embodiments, the system **200** may interact with an application **202** through an operating system. The operating system may include functionality to interact with a file system, which in turn interfaces with a storage pool. The operating system typically interfaces with the file system **200** via a system call interface **208**. The system call interface **208** provides traditional file read, write, open, close, etc., operations, as well as VNODE operations and VFS operations that are specific to the VFS architecture. The system call interface **208** may act as a primary interface for interacting with the ZFS as a file system. This layer resides between a data management unit (DMU) **218** and presents a file system abstraction of the files and directories stored therein. The system call interface **208** may be responsible for bridging the gap between the file system interfaces and the underlying DMU **218** interfaces.

[0051] In addition to the POSIX layer of the system call interface **208**, the interface layer of the file system **200** may also provide a distributed file system interface **210** for interacting with cluster/cloud computing devices **204**. For example, a Lustre® interface may be provided to provide a

file system for computer clusters ranging in size from small workgroup clusters to large-scale, multi-site clusters. A volume emulator **212** may also provide a mechanism for creating logical volumes which can be used as block/character devices. The volume emulator **212** not only allows a client system to distinguish between blocks and characters, but also allows the client system to specify the desired block size and thereby create smaller, sparse volumes in a process known as “thin provisioning.” The volume emulator **212** provides raw access **206** to external devices.

[0052] Underneath the interface layer lies a transactional object layer. This layer provides an intent log **214** configured to record a per-dataset transactional history which can be replayed upon a system crash. In ZFS, the intent log **214** saves transaction records of system calls that change the file system in memory with sufficient information to be able to replay the system calls. These are stored in memory until the DMU **218** commits them to the storage pool and they can be discarded, or they are flushed. In the event of a power failure and/or disk failure, the intent log **214** transactions can be replayed to keep the storage pool up-to-date and consistent.

[0053] The transactional object layer also provides an attribute processor **216** that may be used to implement directories within the POSIX layer of the system call interface **208** by making arbitrary {key, value} associations within an object. The attribute processor **216** may include a module that sits on top of the DMU **218** and may operate on objects referred to in the ZFS as “ZAP objects.” ZAP objects may be used to store properties for a dataset, navigate file system objects, and/or store storage pool properties. ZAP objects may come in two forms: “microzap” objects and “fatzap” objects. Microzap objects may be a lightweight version of the fatzap objects and may provide a simple and fast lookup mechanism for a small number of attribute entries. Fatzap objects may be better suited for ZAP objects containing large numbers of attributes, such as larger directories, longer keys, longer values, etc.

[0054] The transactional object layer also provides a data set and snapshot layer **220** that aggregates DMU objects in a hierarchical namespace and provides a mechanism for describing and managing relationships between properties of object sets. This allows for the inheritance of properties, as well as quota and reservation enforcement in the storage pool. DMU objects may include ZFS file system objects, clone objects, CFS volume objects, and snapshot objects. The data and snapshot layer **220** can therefore manage snapshot and clones.

[0055] A snapshot is a read-only copy of a file system or volume. A snapshot is a view of a filesystem as it was at a particular point in time. ZFS’s snapshots are useful in the same way that some other filesystem’s snapshots are. A backup of a snapshot provides a consistent, non-changing target for the backup program to work with. Snapshots can also be used to recover from recent mistakes, by copying the corrupted files from the snapshot. Snapshots can be created almost instantly, and they initially consume no additional disk space within the pool. However, as data within the active dataset changes, the snapshot consumes disk space by continuing to reference the old data, thus preventing the disk space from being freed. The blocks containing the old data may only be freed if the snapshot is deleted. Taking a snapshot is a constant-time operation. The presence of snapshots does not slow down any operations. Deleting snapshots takes time proportional to the number of blocks

that the delete will free and can be very efficient. ZFS snapshots can include the following features: they persist across system reboots; the theoretical maximum number of snapshots is 264; they use no separate backing store; they consume disk space directly from the same storage pool as the file system or volume from which they were created; recursive snapshots are created quickly as one atomic operation; and they are created together (all at once) or not created at all. The benefit of atomic snapshot operations is that the snapshot data is always taken at one consistent time, even across descendent file systems. Snapshots cannot be accessed directly, but they can be cloned, backed up, rolled back to, and so on. Snapshots can be used to “roll back” in time to the point when the snapshot was taken

[0056] A clone is a writable volume or file system whose initial contents are the same as the dataset from which it was created. In the ZFS system clones can be created from snapshots. As with snapshots, creating a clone is nearly instantaneous and initially consumes no additional disk space. In addition, a system can generate a snapshot of a clone. Clones can also be created from a snapshot. When a snapshot is cloned, an implicit dependency is created between the clone and snapshot. Even though the clone is created somewhere else in the dataset hierarchy, the original snapshot cannot be destroyed as long as the clone exists. Clones do not inherit the properties of the dataset from which it was created. A clone initially shares all its disk space with the original snapshot. As changes are made to the clone, it uses more disk space. Clones are useful to branch off and do development or troubleshooting—and can be promoted to replace the live file system. Clones can also be used to duplicate a file system on multiple machines.

[0057] The DMU 218 presents a transactional object model built on top of a flat address space presented by the storage pool. The modules described above interact with the DMU 218 via object sets, objects, and transactions, where objects are pieces of storage from the storage pool, such as a collection of data blocks. Each transaction through the DMU 218 comprises a series of operations that are committed to the storage pool as a group. This is the mechanism whereby on-disk consistency is maintained within the file system. Stated another way, the DMU 218 takes instructions from the interface layer and translates those into transaction batches. Rather than requesting data blocks and sending single read/write requests, the DMU 218 can combine these into batches of object-based transactions that can be optimized before any disk activity occurs. Once this is done, the batches of transactions are handed off to the storage pool layer to schedule and aggregate the raw I/O transactions required to retrieve/write the requested data blocks. As will be described below, these transactions are written on a copy-on-write (COW) basis, which eliminates the need for transaction journaling.

[0058] The storage pool layer, or simply the “storage pool,” may be referred to as a storage pool allocator (SPA). The SPA provides public interfaces to manipulate storage pool configuration. These interfaces can create, destroy, import, export, and pool various storage media and manage the namespace of the storage pool. In some embodiments, the SPA may include an adaptive replacement cache (ARC) 222 that acts as a central point for memory management for the SPA. Traditionally, an ARC provides a basic least-recently-used (LRU) object replacement algorithm for cache management. In ZFS, the ARC 222 comprises a self-tuning

cache that can adjust based on the I/O workload. Additionally, the ARC 222 defines a data virtual address (DVA) that is used by the DMU 218. In some embodiments, the ARC 222 has the ability to evict memory buffers from the cache as a result of memory pressure to maintain a high throughput.

[0059] The SPA may also include an I/O pipeline 224, or “I/O manager,” that translates the DVAs from the ARC 222 into logical locations in each of the virtual devices (VDEVs) 226 described below. The I/O pipeline 224 drives the dynamic striping, compression, checksum capabilities, and data redundancy across the active VDEVs. Although not shown explicitly in FIG. 2, the I/O pipeline 224 may include other modules that may be used by the SPA to read data from and/or write data to the storage pool. For example, the I/O pipeline 224 may include, without limitation, a compression module, an encryption module, a checksum module, and a metaslab allocator. The checksum may be used, for example, to ensure data has not been corrupted. In some embodiments, the SPA may use the metaslab allocator to manage the allocation of storage space in the storage pool.

[0060] Compression is the process of reducing the data size of a data block (referred to interchangeably with leaf node or data node), typically by exploiting redundancies in the data block itself. Many different compression types are used by ZFS. When compression is enabled, less storage can be allocated for each data block. The following compression algorithms are available. LZ4—an algorithm added after feature flags were created. It is significantly superior to LZJB. LZJB is the original default compression algorithm) for ZFS. It was created to satisfy the desire for a compression algorithm suitable for use in filesystems. Specifically, that it provides fair compression, has a high compression speed, has a high decompression speed, and detects incompressible data detection quickly. GZIP (1 through 9 implemented in the classic Lempel-Ziv implementation. It provides high compression, but it often makes IO CPU-bound. ZLE (Zero Length Encoding)—a very simple algorithm that only compresses zeroes. In each of these cases there is a trade-off of compression ratio to the amount of latency involved in compressing and decompressing the data block. Typically—the more compressed the data—the longer it takes to compress and decompress it.

[0061] Encryption is the process of adding end-to-end security to data blocks by encoding them cryptographically with a key. Only users with a key can decrypt the data block. As used in the ZFS system, a ZFS pool can support a mix of encrypted and unencrypted ZFS data sets (file systems and ZVOLs). Data encryption is completely transparent to applications and provides a very flexible system for securing data at rest, and it doesn’t require any application changes or qualification. Furthermore, ZFS encryption randomly generates a local encryption key from a passphrase, or an AES key and all keys are stored locally with the client—not in the cloud object store 404 as traditional file systems do. Encryption is transparent to the application and storage to the cloud object store 404 when turned on. ZFS makes it easy to encrypt data and manage data encryption. You can have both encrypted and unencrypted file systems in the same storage pool. You can also use different encryption keys for different systems, and you can manage encryption either locally or remotely—although the randomly generated encryption key always remains local. ZFS encryption is inheritable to descendent file systems. Data is encrypted using AES (Ad-

vanced Encryption Standard) with key lengths of **128**, **192**, and **256** in the CCM and GCM operation modes.

[0062] Deduplication is the process of recognizing that a data block to be stored in the file system is already stored on the file system as an existing data block and pointing to that existing data block rather than storing the data block again. ZFS provides block-level deduplication because this is the finest granularity that makes sense for a general-purpose storage system. Block-level dedup also maps naturally to ZFS's 256-bit block checksums, which provide unique block signatures for all blocks in a storage pool as long as the checksum function is cryptographically strong (e.g., SHA256). Deduplication is synchronous and can be performed as data blocks are sent to the cloud object store **404**. If data blocks are not duplicated, enabling deduplication will add overhead without providing any benefit. If there are duplicate data blocks, enabling deduplication will both save space and increase performance. The space savings are obvious; the performance improvement is due to the elimination of storage writes when storing duplicate data, plus the reduced memory footprint due to many applications sharing the same pages of memory. Most storage environments contain a mix of data that is mostly unique and data that is mostly replicated. ZFS deduplication is per-dataset and can be enabled when it is likely to help.

[0063] In ZFS, the storage pools may be made up of a collection of VDEVs. In certain embodiments, at least a portion of the storage pools may be represented as a self-described Merkle tree, a logical tree where both data and metadata are stored by VDEVs of the logical tree.

[0064] There are two types of virtual devices: physical virtual devices called leaf VDEVs, and logical virtual devices called interior VDEVs. A physical VDEV may include a writeable media block device, such as a hard disk or Flash drive. A logical VDEV is a conceptual grouping of physical VDEVs. VDEVs can be arranged in a tree with physical VDEVs existing as leaves of the tree. The storage pool may have a special logical VDEV called a "root VDEV" which roots the tree. All direct children of the root VDEV (physical or logical) are called "top-level" VDEVs. In general, VDEVs implement data replication, mirroring, and architectures such as RAID-Z and RAID-Z2. Each leaf VDEV represents one or more physical storage devices **228** that actually store the data provided by the file system.

[0065] In some embodiments, the file system **200** may include an object-based file system where both data and metadata are stored as objects. More specifically, the file system **200** may include functionality to store both data and corresponding metadata in the storage pool. A request to perform a particular operation (i.e., a transaction) is forwarded from the operating system, via the system call interface **208**, to the DMU **218**, which translates the request to perform an operation on an object directly to a request to perform a read or write operation (i.e., an I/O request) at a physical location within the storage pool. The SPA receives the request from the DMU **218** and writes the blocks into the storage pool using a COW procedure. COW transactions may be performed for a data write request to a file. Instead of overwriting existing blocks on a write operation, write requests cause new segments to be allocated for the modified data. Thus, retrieved data blocks and corresponding metadata are never overwritten until a modified version of the data block and metadata are committed. Thus, the DMU **218** writes all the modified data blocks to unused segments

within the storage pool and subsequently writes corresponding block pointers to unused segments within the storage pool. To complete a COW transaction, the SPA issues an I/O request to reference the modified data block.

[0066] FIGS. 3A-3D illustrate a COW process for a file system, such as the file system **200**, in accordance with certain embodiments of the present disclosure. For example, the ZFS system described above uses a COW transactional model where all block pointers within the file system may contain 256-bit checksum of a target block which is verified when the block is read. As described above, blocks containing active data are not overwritten in place. Instead, the new block is allocated, modified data is written to it, and then any metadata blocks referencing it are simply read, reallocated, and rewritten.

[0067] FIG. 3A illustrates a simplified diagram of a file system storage of data and metadata corresponding to one or more files as a logical tree **300**, according to some embodiments. The logical tree **300**, as well as other logical trees described herein, may be a self-described Merkle tree where the data and metadata are stored as blocks of the logical tree **300**. A root block **302** may represent the root of the logical tree **300**, or "uberblock." The logical tree **300** can be traversed through files and directories by navigating through each child node **304**, **306** of the root **302**. Each non-leaf node represents a directory or file, such as nodes **308**, **310**, **312**, and **314**. In some embodiments, each non-leaf node may be assigned a hash of values of its child nodes. Each leaf node **316**, **318**, **320**, **322** represents a data block of a file.

[0068] FIG. 3B illustrates an example of the logical tree **300-1** after an initial stage of a write operation. In this example, the data blocks represented by nodes **324** and **326** have been written by the file system **200**. Instead of overwriting the data in nodes **316** and **318**, new data blocks are allocated for nodes **324** and **326**. Thus, after this operation, the old data in nodes **316** and **318** persist in the memory along with the new data in nodes **324** and **326**.

[0069] FIG. 3C illustrates an example of the logical tree **300-2** as the write operation continues. In order to reference the newly written data blocks in nodes **324** and **326**, the file system **200** determines nodes **308** and **310** that reference the old nodes **316** and **318**. New nodes **328** and **330** are allocated to reference the new data blocks in nodes **324** **326**. The same process is repeated recursively upwards through the file system hierarchy until each node referencing a changed node is reallocated to point to the new nodes.

[0070] When the pointer blocks are allocated in new nodes in the hierarchy, the address pointer in each node is updated to point to the new location of the allocated child in memory. Additionally, each data block includes a checksum that is calculated by the data block referenced by the address pointer. For example, the checksum in node **328** is calculated using the data block in node **324**. This arrangement means that the checksum is stored separately from the data block from which it is calculated. This prevents so-called "ghost writes" where new data is never written, but a checksum stored with the data block would indicate that the block was correct. The integrity of the logical tree **300** can be quickly checked by traversing the logical tree **300** and calculating checksums at each level based on child nodes.

[0071] In order to finalize the write operation, the root **302** can be reallocated and updated. FIG. 3D illustrates an example of the logical tree **300-3** at the conclusion of the write operation. When the root **302** is ready to be updated,

a new uberblock root **336** can be allocated and initialized to point to the newly allocated child nodes **332** and **334**. The root **336** can then be made the root of the logical tree **300-3** in an atomic operation to finalize the state of the logical tree **300-3**.

[0072] A snapshot is a read-only copy of a file system or volume. A snapshot is a view of a filesystem as it was at a particular point in time. ZFS's snapshots can be useful in the same way that some other file systems' snapshots. By doing a backup of a snapshot, the system can have a consistent, non-changing target for the backup program to work with. Snapshots can also be used to recover from recent mistakes, by copying the corrupted files from the snapshot. Snapshots can be created almost instantly, and they initially consume no additional disk space within the pool. However, as data within the active dataset changes, the snapshot consumes disk space by continuing to reference the old data, thus preventing the disk space from being freed. The blocks containing the old data will only be freed if the snapshot is deleted. Taking a snapshot is a constant-time operation. The presence of snapshots does not slow down any operations. Deleting snapshots takes time proportional to the number of blocks that the delete will free and is very efficient. ZFS snapshots include the following features: they persist across system reboots; the theoretical maximum number of snapshots is 264; they use no separate backing store; they consume disk space directly from the same storage pool as the file system or volume from which they were created; recursive snapshots are created quickly as one atomic operation; and they are created together (all at once) or not created at all. The benefit of atomic snapshot operations is that the snapshot data is always taken at one consistent time, even across descendent file systems. Snapshots cannot be accessed directly, but they can be cloned, backed up, rolled back to, and so on. Snapshots can be used to "roll back" in time to the point when the snapshot was taken. FIG. 3E depicts an example of the snapshot data service in ZFS where the snapshot was taken before the COW process described in FIGS. 3A-3D made root block **336** the new live root block. A live root block is the root block that the next data progression will be made from when performing a COW. The snapshot root and the "live" root are shown. The live root is the root that will be operated on in the next storage operation. All blocks pointed to by the snapshot root (**302-322**) are made "read only" meaning that they are put on list of blocks that cannot be freed for further use by the storage system until the snapshot is deleted.

[0073] It should be noted that some terminology is used interchangeably throughout the application. For instance, leaf nodes, leaf blocks, and data blocks may be the same in certain instances, particularly when referencing local tree instances. Further, non-leaf nodes, metadata, and metadata blocks may be used interchangeably in certain instances, particularly when referencing local tree instances. Root nodes and root blocks similarly may be used interchangeably in certain instances, particularly when referencing local tree instances. Further, it should be noted that references to leaf nodes, non-leaf nodes, and root nodes may be similarly applied to cloud storage objects corresponding to cloud versions of logical trees generated based at least in part on local tree instances.

[0074] When blocks are created, they are given a "birth" time that represents the iteration or progression of the COW that the block was created. FIG. 3F demonstrates this idea.

In FIG. 3F, as shown in **365**—the birth time **19** of FIG. 3A. **366** shows the birth time, **25** of FIG. 3D. **367** shows a birth time **37** as shown by the new tree created by blocks **372**, **378**, **379**, **384**, **386**, and **392**, and represents a data transaction on the tree **12** iterations after birth time **25**. Thus, a rollback or back up to the snapshot would leave only the blocks as shown in FIG. 3A. Thus—using a birth time hierarchy—the ZFS system can generate and roll back to any point in a birth time for the entire tree structure from a snapshot of the root of the tree. Essentially, this allows all new blocks with birth times after the snapshot to be made available in the storage pool as long as they are not linked to by any other snapshot or metadata block.

[0075] A clone is a writable volume or file system whose initial contents are the same as the dataset from which it was created. In the ZFS system, clones are always created from snapshots. As with snapshots, creating a clone is nearly instantaneous and initially consumes no additional disk space. In addition, you can snapshot a clone. Clones can only be created from a snapshot. When a snapshot is cloned, an implicit dependency is created between the clone and snapshot. Even though the clone is created somewhere else in the dataset hierarchy, the original snapshot cannot be destroyed as long as the clone exists. Clones do not inherit the properties of the dataset from which it was created. A clone initially shares all its disk space with the original snapshot. As changes are made to the clone, it uses more disk space. Clones are useful to branch off and do development or troubleshooting—and can be promoted to replace the live file system. Clones can also be used to duplicate a file system on multiple machines.

[0076] The embodiments described herein may be implemented in the system described above in FIGS. 1-3. For example, the system may comprise one or more processors of the various servers, storage appliances, and/or switching circuits of FIG. 1. Instructions may be stored in one or more memory devices of the system that cause the one or more processors to perform various operations that affect the functioning of the file system. Steps of various methods may be performed by the processors, memory devices, interfaces, and/or circuitry of the system in FIGS. 1-2.

[0077] FIG. 4 illustrates an exemplary network file system **400**. The network file system **400** can include a head node **402** that can list one or more directories (e.g., Directory-A **404**). Directory-A **404** can include one or more subfolders (e.g., Subfolder-A **408**, Subfolder-B **410**, and Subfolder-C **412**). The directory can have any number of subfolders and the three subfolders are merely exemplary. Each of the subfolders can store a plurality of files. For example, Subfolder-A **408** can store File-A1 **422**, File-A2 **424**, and File-A3 **426**. Subfolder-B **410** can store File-B1 **428**, File-B2 **430**, and File-B3 **432**. Subfolder-C **412** can store File-C1 **434**, File-C2 **436**, and File-C3 **438**. While three number of files is illustrated in FIG. 4 for each of the subfolders any number of files can be stored in either the root directory or the subfolders.

[0078] A user may desire to create a clone Directory-A **404** including the subfolders and the stored files. The system can generate a clone as Directory-A' **406**. Clone Directory-A' **406** can include one or more subfolders (e.g., Subfolder-A' **414**, Subfolder-B' **416** and Subfolder-C' **4418**). The clone directory can have any number of subfolders and the three subfolders are merely exemplary. The clone directory can duplicate all the subfolders and files of the target directory

(e.g., Directory-A' 406). Each of the subfolders can store a plurality of files. For example, Subfolder-A' 414 can store File-A1' 440, File-A2' 442, and File-A3' 444. Subfolder-B' 416 can store File-B1' 446, File-B2' 448, and File-B3' 450. Subfolder-C' 418 can store File-C1' 452, File-C2' 454, and File-C3' 456. Clone Directory-A' 406 can be listed on the head node so a client can have immediate access to the clone directory and the files stored therein.

[0079] The advantages of the techniques described herein is that the clone directory can be immediately available to clients and customers. The clone directory can be made visible in the head node. This also makes the management of the file directories easier because there will not be an explosion of mount points on the client. Instead of having 11 mount points for ten clones and one head, these techniques provide one mount point for the head node that includes all of its clones. Previously, for a customer to access the clone directory, a message needed to be sent out and an administrator would need to notify the clients that the clone directory was ready to be mounted for use. The techniques basically trick NFS into believing that all of these snapshots and these clones are all part of the head dataset so they can all be seen from the same NFS mount point.

[0080] FIG. 5 is a flow chart of a process 500 for automatic filesystem clone generation, according to an example of the present disclosure. According to an example, one or more process blocks of FIG. 5 may be performed by a computing system.

[0081] At block 505, process 500 may include receiving a selection indicating that a clone directory is visible in a head node. The clone directory can be visible or not visible in the head node. The clone directory may not be accessible by the clients if it is not made visible in the head node. For example, computing system may receive a selection indicating that a clone directory is visible in a head node, as described above. The selection can be made on a touch-screen display or an input from a pointing device (e.g., a trackball, a mouse, etc.)

[0082] At block 510, in response to the selection indicating that the clone directory to be visible in the head node, process 500 may include generating a snapshot of a target file system. For example, computing system may generate a snapshot of a target file system, as described above. In various embodiments, files in the clone directory are read-write files.

[0083] At block 515, process 500 may include duplicating a directory of the target file system into the clone directory. For example, computing system may duplicate a directory of the target file system into the clone directory, as described above.

[0084] At block 520, process 500 may include mapping an address of the target file system into the clone directory. For example, computing system may map an address of the target file system into the clone directory, as described above. This can turn the snapshot into a live mounted and shared read/write copy of the snapshot immediately. In various embodiments, the process 500 can map existing snapshots that were previously created. In various embodiments, a "mkdir" command can be used to create a snapshot and clone of the filesystem in its current state. In various embodiments, a link command ("ln") command can be used to create a clone of an existing snapshot.

[0085] At block 525, process 500 may include linking the clone directory to the head node. For example, computing

system may link the clone directory to the head node, as described above. The linking allows the clone directory to be visible in the head node.

[0086] This allows people or processes with filesystem access the ability to duplicate their filesystem using what is normally only provided to an administrator on an administrative interface by a simple filesystem command. That simple "mkdir" command can use a low-overhead mechanism to create that duplicate writable filesystem providing near instant access to a writable copy of the original filesystem.

[0087] This process can be built upon the existing ZFS features of snapshots and clones, including the "snapdir" feature which is used to create snapshots using "mkdir" in a special directory.

[0088] Essentially, this process 500 can provides a second special directory named "clonedir" which, when "mkdir" is run on it, a snapshot is taken of the filesystem it is in, and that snapshot cloned to an active read-write mounted filesystem named after the directory specified in the mkdir command and mounted there. If, instead, a link is created using "ln" from the snapdir directory to the new clonedir directory, that existing snapshot is cloned and mounted as a read-write filesystem under the name specified in the link command.

[0089] Process 500 may include additional implementations, such as any single implementation or any combination of implementations described below and/or in connection with one or more other processes described elsewhere herein. A first implementation, process 500 may include mirroring security privileges and administrative access from the target file system to the clone directory.

[0090] In various embodiments, process 500 may include receiving a selection of the snapshot of the target file system that is stored in a snapshot directory.

[0091] In various embodiments, the snapshot is a point-in-time reference of the target file system.

[0092] In various embodiments, process 500 may include copying file system properties of the target file system, where the file system properties include one or more of compartment, tags, display name, keys, and mount target export information.

[0093] In various embodiments, process 500 may include copying linking metadata from the target file system to the clone directory.

[0094] It should be noted that while FIG. 5 shows example blocks of process 500, in some implementations, process 500 may include additional blocks, fewer blocks, different blocks, or differently arranged blocks than those depicted in FIG. 5. Additionally, or alternatively, two or more of the blocks of process 500 may be performed in parallel.

[0095] FIG. 6 depicts a simplified diagram of a distributed system 600 for implementing certain embodiments in accordance with present disclosure. In the illustrated embodiment, distributed system 600 includes one or more client computing devices 602, 604, 606, and 608, which are configured to execute and operate a client application such as a web browser, proprietary client (e.g., Oracle Forms), or the like over one or more network(s) 610. Server 612 may be communicatively coupled with remote client computing devices 602, 604, 606, and 608 via network 610.

[0096] In various embodiments, server 612 may be adapted to run one or more services or software applications provided by one or more of the components of the system.

In some embodiments, these services may be offered as web-based or cloud services or under a Software as a Service (SaaS) model to the users of client computing devices **602**, **604**, **606**, and/or **608**. Users operating client computing devices **602**, **604**, **606**, and/or **608** may in turn utilize one or more client applications to interact with server **612** to utilize the services provided by these components.

[0097] In the configuration depicted in the figure, the software components **618**, **620** and **622** of system **600** are shown as being implemented on server **612**. In other embodiments, one or more of the components of system **600** and/or the services provided by these components may also be implemented by one or more of the client computing devices **602**, **604**, **606**, and/or **608**. Users operating the client computing devices may then utilize one or more client applications to use the services provided by these components. These components may be implemented in hardware, firmware, software, or combinations thereof. It should be appreciated that various different system configurations are possible, which may be different from distributed system **600**. The embodiment shown in the figure is thus one example of a distributed system for implementing an embodiment system and is not intended to be limiting.

[0098] Client computing devices **602**, **604**, **606**, and/or **608** may be portable handheld devices (e.g., an iPhone®, cellular telephone, an iPad®, computing tablet, a personal digital assistant (PDA)) or wearable devices (e.g., a Google Glass® head-mounted display), running software such as Microsoft Windows Mobile®, and/or a variety of mobile operating systems such as iOS, Windows Phone, Android, BlackBerry, Palm OS, and the like, and being Internet, e-mail, short message service (SMS), BlackBerry®, or other communication protocol enabled. The client computing devices can be general purpose personal computers including, by way of example, personal computers and/or laptop computers running various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems. The client computing devices can be workstation computers running any of a variety of commercially available UNIX® or UNIX-like operating systems, including without limitation the variety of GNU/Linux operating systems, such as for example, Google Chrome OS. Alternatively, or in addition, client computing devices **602**, **604**, **606**, and **608** may be any other electronic device, such as a thin-client computer, an Internet-enabled gaming system (e.g., a Microsoft Xbox gaming console with or without a Kinect® gesture input device), and/or a personal messaging device, capable of communicating over network(s) **610**.

[0099] Although exemplary distributed system **600** is shown with four client computing devices, any number of client computing devices may be supported. Other devices, such as devices with sensors, etc., may interact with server **612**.

[0100] Network(s) **610** in distributed system **600** may be any type of network familiar to those skilled in the art that can support data communications using any of a variety of commercially-available protocols, including without limitation TCP/IP (transmission control protocol/Internet protocol), SNA (systems network architecture), IPX (Internet packet exchange), AppleTalk, and the like. Merely by way of example, network(s) **610** can be a local area network (LAN), such as one based on Ethernet, Token-Ring and/or the like. Network(s) **610** can be a wide-area network and the Internet. It can include a virtual network, including without limitation

a virtual private network (VPN), an intranet, an extranet, a public switched telephone network (PSTN), an infra-red network, a wireless network (e.g., a network operating under any of the Institute of Electrical and Electronics (IEEE) 802.11 suite of protocols, Bluetooth®, and/or any other wireless protocol); and/or any combination of these and/or other networks.

[0101] Server **612** may be composed of one or more general purpose computers, specialized server computers (including, by way of example, PC (personal computer) servers, UNIX® servers, mid-range servers, mainframe computers, rack-mounted servers, etc.), server farms, server clusters, or any other appropriate arrangement and/or combination. In various embodiments, server **612** may be adapted to run one or more services or software applications described in the foregoing disclosure. For example, server **612** may correspond to a server for performing processing described above according to an embodiment of the present disclosure.

[0102] Server **612** may run an operating system including any of those discussed above, as well as any commercially available server operating system. Server **612** may also run any of a variety of additional server applications and/or mid-tier applications, including HTTP (hypertext transport protocol) servers, FTP (file transfer protocol) servers, CGI (common gateway interface) servers, JAVA® servers, database servers, and the like. Exemplary database servers include without limitation those commercially available from Oracle, Microsoft, Sybase, IBM (International Business Machines), and the like.

[0103] In some implementations, server **612** may include one or more applications to analyze and consolidate data feeds and/or event updates received from users of client computing devices **602**, **604**, **606**, and **608**. As an example, data feeds and/or event updates may include, but are not limited to, Twitter® feeds, Facebook® updates or real-time updates received from one or more third party information sources and continuous data streams, which may include real-time events related to sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), click-stream analysis tools, automobile traffic monitoring, and the like. Server **612** may also include one or more applications to display the data feeds and/or real-time events via one or more display devices of client computing devices **602**, **604**, **606**, and **608**.

[0104] Distributed system **600** may also include one or more databases **614** and **616**. Databases **614** and **616** may reside in a variety of locations. By way of example, one or more of databases **614** and **616** may reside on a non-transitory storage medium local to (and/or resident in) server **612**. Alternatively, databases **614** and **616** may be remote from server **612** and in communication with server **612** via a network-based or dedicated connection. In one set of embodiments, databases **614** and **616** may reside in a storage-area network (SAN). Similarly, any necessary files for performing the functions attributed to server **612** may be stored locally on server **612** and/or remotely, as appropriate. In one set of embodiments, databases **614** and **616** may include relational databases, such as databases provided by Oracle, which are adapted to store, update, and retrieve data in response to SQL-formatted commands.

[0105] FIG. 7 is a simplified block diagram of one or more components of a system environment **700** by which services

provided by one or more components of a system may be offered as cloud services, in accordance with certain embodiments of the present disclosure. In the illustrated embodiment, system environment 700 includes one or more client computing devices 704, 706, and 708 that may be used by users to interact with a cloud infrastructure system 702 that provides cloud services. The client computing devices may be configured to operate a client application such as a web browser, a proprietary client application (e.g., Oracle Forms), or some other application, which may be used by a user of the client computing device to interact with cloud infrastructure system 702 to use services provided by cloud infrastructure system 702.

[0106] It should be appreciated that cloud infrastructure system 702 depicted in the figure may have other components than those depicted. Further, the embodiment shown in the figure is only one example of a cloud infrastructure system that may incorporate an embodiment of the invention. In some other embodiments, cloud infrastructure system 702 may have more or fewer components than shown in the figure, may combine two or more components, or may have a different configuration or arrangement of components.

[0107] Client computing devices 704, 706, and 708 may be devices similar to those described above for 602, 604, 606, and 608. Although exemplary system environment 700 is shown with three client computing devices, any number of client computing devices may be supported. Other devices such as devices with sensors, etc. may interact with cloud infrastructure system 702.

[0108] Network(s) 710 may facilitate communications and exchange of data between clients 704, 706, and 708 and cloud infrastructure system 702. Each network may be any type of network familiar to those skilled in the art that can support data communications using any of a variety of commercially available protocols, including those described above for network(s) 610. Cloud infrastructure system 702 may comprise one or more computers and/or servers that may include those described above for server 612.

[0109] In certain embodiments, services provided by the cloud infrastructure system may include a host of services that are made available to users of the cloud infrastructure system on demand, such as online data storage and backup solutions, Web-based e-mail services, hosted office suites and document collaboration services, database processing, managed technical support services, and the like. Services provided by the cloud infrastructure system can dynamically scale to meet the needs of its users. A specific instantiation of a service provided by cloud infrastructure system is referred to herein as a “service instance.” In general, any service made available to a user via a communication network, such as the Internet, from a cloud service provider’s system is referred to as a “cloud service.” Typically, in a public cloud environment, servers and systems that make up the cloud service provider’s system are different from the customer’s own on-premises servers and systems. For example, a cloud service provider’s system may host an application, and a user may, via a communication network such as the Internet, on demand, order and use the application.

[0110] In some examples, a service in a computer network cloud infrastructure may include protected computer network access to storage, a hosted database, a hosted web server, a software application, or other service provided by

a cloud vendor to a user, or as otherwise known in the art. For example, a service can include password-protected access to remote storage on the cloud through the Internet. As another example, a service can include a web service-based hosted relational database and a script-language middleware engine for private use by a networked developer. As another example, a service can include access to an email software application hosted on a cloud vendor’s web site.

[0111] In certain embodiments, cloud infrastructure system 702 may include a suite of applications, middleware, and database service offerings that are delivered to a customer in a self-service, subscription-based, elastically scalable, reliable, highly available, and secure manner. An example of such a cloud infrastructure system is the Oracle Public Cloud provided by the present assignee.

[0112] In various embodiments, cloud infrastructure system 702 may be adapted to automatically provision, manage and track a customer’s subscription to services offered by cloud infrastructure system 702. Cloud infrastructure system 702 may provide the cloud services via different deployment models. For example, services may be provided under a public cloud model in which cloud infrastructure system 702 is owned by an organization selling cloud services (e.g., owned by Oracle) and the services are made available to the general public or different industry enterprises. As another example, services may be provided under a private cloud model in which cloud infrastructure system 702 is operated solely for a single organization and may provide services for one or more entities within the organization. The cloud services may also be provided under a community cloud model in which cloud infrastructure system 702 and the services provided by cloud infrastructure system 702 are shared by several organizations in a related community. The cloud services may also be provided under a hybrid cloud model, which is a combination of two or more different models.

[0113] In some embodiments, the services provided by cloud infrastructure system 702 may include one or more services provided under Software as a Service (SaaS) category, Platform as a Service (PaaS) category, Infrastructure as a Service (IaaS) category, or other categories of services including hybrid services. A customer, via a subscription order, may order one or more services provided by cloud infrastructure system 702. Cloud infrastructure system 702 then performs processing to provide the services in the customer’s subscription order.

[0114] In some embodiments, the services provided by cloud infrastructure system 702 may include, without limitation, application services, platform services and infrastructure services. In some examples, application services may be provided by the cloud infrastructure system via a SaaS platform. The SaaS platform may be configured to provide cloud services that fall under the SaaS category. For example, the SaaS platform may provide capabilities to build and deliver a suite of on-demand applications on an integrated development and deployment platform. The SaaS platform may manage and control the underlying software and infrastructure for providing the SaaS services. By utilizing the services provided by the SaaS platform, customers can utilize applications executing on the cloud infrastructure system. Customers can acquire the application services without the need for customers to purchase separate licenses and support. Various different SaaS services may be pro-

vided. Examples include, without limitation, services that provide solutions for sales performance management, enterprise integration, and business flexibility for large organizations.

[0115] In some embodiments, platform services may be provided by the cloud infrastructure system via a PaaS platform. The PaaS platform may be configured to provide cloud services that fall under the PaaS category. Examples of platform services may include without limitation services that enable organizations (such as Oracle) to consolidate existing applications on a shared, common architecture, as well as the ability to build new applications that leverage the shared services provided by the platform. The PaaS platform may manage and control the underlying software and infrastructure for providing the PaaS services. Customers can acquire the PaaS services provided by the cloud infrastructure system without the need for customers to purchase separate licenses and support. Examples of platform services include, without limitation, Oracle Java Cloud Service (JCS), Oracle Database Cloud Service (DBCS), and others.

[0116] By utilizing the services provided by the PaaS platform, customers can employ programming languages and tools supported by the cloud infrastructure system and also control the deployed services. In some embodiments, platform services provided by the cloud infrastructure system may include database cloud services, middleware cloud services (e.g., Oracle Fusion Middleware services), and Java cloud services. In one embodiment, database cloud services may support shared service deployment models that enable organizations to pool database resources and offer customers a Database as a Service in the form of a database cloud. Middleware cloud services may provide a platform for customers to develop and deploy various business applications, and Java cloud services may provide a platform for customers to deploy Java applications, in the cloud infrastructure system.

[0117] Various different infrastructure services may be provided by an IaaS platform in the cloud infrastructure system. The infrastructure services facilitate the management and control of the underlying computing resources, such as storage, networks, and other fundamental computing resources for customers utilizing services provided by the SaaS platform and the PaaS platform.

[0118] In certain embodiments, cloud infrastructure system 702 may also include infrastructure resources 730 for providing the resources used to provide various services to customers of the cloud infrastructure system. In one embodiment, infrastructure resources 730 may include pre-integrated and optimized combinations of hardware, such as servers, storage, and networking resources to execute the services provided by the PaaS platform and the SaaS platform. In some embodiments, resources in cloud infrastructure system 702 may be shared by multiple users and dynamically re-allocated per demand. Additionally, resources may be allocated to users in different time zones. For example, cloud infrastructure system 730 may enable a first set of users in a first time zone to utilize resources of the cloud infrastructure system for a specified number of hours and then enable the re-allocation of the same resources to another set of users located in a different time zone, thereby maximizing the utilization of resources.

[0119] In certain embodiments, a number of internal shared services 732 may be provided that are shared by different components or modules of cloud infrastructure

system 702 and by the services provided by cloud infrastructure system 702. These internal shared services may include, without limitation, a security and identity service, an integration service, an enterprise repository service, an enterprise manager service, a virus scanning and whitelist service, a high availability, backup and recovery service, service for enabling cloud support, an email service, a notification service, a file transfer service, and the like. In certain embodiments, cloud infrastructure system 702 may provide comprehensive management of cloud services (e.g., SaaS, PaaS, and IaaS services) in the cloud infrastructure system. In one embodiment, cloud management functionality may include capabilities for provisioning, managing, and tracking a customer's subscription received by cloud infrastructure system 702, and the like.

[0120] In certain embodiments, as depicted in the figure, cloud management functionality may be provided by one or more modules, such as an order management module 720, an order orchestration module 722, an order provisioning module 724, an order management and monitoring module 726, and an identity management module 728. These modules may include or be provided using one or more computers and/or servers, which may be general purpose computers, specialized server computers, server farms, server clusters, or any other appropriate arrangement and/or combination.

[0121] In exemplary operation 734, a customer using a client device, such as client device 704, 706 or 708, may interact with cloud infrastructure system 702 by requesting one or more services provided by cloud infrastructure system 702 and placing an order for a subscription for one or more services offered by cloud infrastructure system 702. In certain embodiments, the customer may access a cloud User Interface (UI), cloud UI 712, cloud UI 714 and/or cloud UI 716 and place a subscription order via these UIs. The order information received by cloud infrastructure system 702 in response to the customer placing an order may include information identifying the customer and one or more services offered by the cloud infrastructure system 702 that the customer intends to subscribe to.

[0122] After an order has been placed by the customer, the order information is received via the cloud UIs, 712, 714 and/or 716. At operation 736, the order is stored in order database 718. Order database 718 can be one of several databases operated by a cloud infrastructure system and operated in conjunction with other system elements. At operation 738, the order information is forwarded to an order management module 720. In some instances, order management module 720 may be configured to perform billing and accounting functions related to the order, such as verifying the order, and upon verification, booking the order.

[0123] At operation 740, information regarding the order is communicated to an order orchestration module 722. Order orchestration module 722 may utilize the order information to orchestrate the provisioning of services and resources for the order placed by the customer. In some instances, order orchestration module 722 may orchestrate the provisioning of resources to support the subscribed services using the services of order provisioning module 724.

[0124] In certain embodiments, order orchestration module 722 enables the management of business processes associated with each order and applies business logic to determine whether an order should proceed to provisioning.

At operation **742**, upon receiving an order for a new subscription, order orchestration module **722** sends a request to order provisioning module **724** to allocate resources and configure those resources needed to fulfill the subscription order. Order provisioning module **724** enables the allocation of resources for the services ordered by the customer. Order provisioning module **724** provides a level of abstraction between the cloud services provided by cloud infrastructure system **700** and the physical implementation layer that is used to provision the resources for providing the requested services. Order orchestration module **722** may thus be isolated from implementation details, such as whether or not services and resources are actually provisioned on the fly or pre-provisioned and only allocated/assigned upon request.

[0125] At operation **744**, once the services and resources are provisioned, a notification of the provided service may be sent to customers on client devices **704**, **706**, and/or **708** by order provisioning module **724** of cloud infrastructure system **702**. At operation **746**, the customer's subscription order may be managed and tracked by an order management and monitoring module **726**. In some instances, order management and monitoring module **726** may be configured to collect usage statistics for the services in the subscription order, such as the amount of storage used, the amount data transferred, the number of users, and the amount of system up time and system down time.

[0126] In certain embodiments, cloud infrastructure system **700** may include an identity management module **728**. Identity management module **728** may be configured to provide identity services, such as access management and authorization services in cloud infrastructure system **700**. In some embodiments, identity management module **728** may control information about customers who wish to utilize the services provided by cloud infrastructure system **702**. Such information can include information that authenticates the identities of such customers and information that describes which actions those customers are authorized to perform relative to various system resources (e.g., files, directories, applications, communication ports, memory segments, etc.). Identity management module **728** may also include the management of descriptive information about each customer and about how and by whom that descriptive information can be accessed and modified.

[0127] FIG. **8** illustrates an exemplary computer system **800**, in which various embodiments of the present invention may be implemented. The system **800** may be used to implement any of the computer systems described herein. As shown in the figure, computer system **800** includes a processing unit **804** that communicates with a number of peripheral subsystems via a bus subsystem **802**. These peripheral subsystems may include a processing acceleration unit **806**, an I/O subsystem **808**, a storage subsystem **818** and a communications subsystem **824**. Storage subsystem **818** includes tangible computer-readable storage media **822** and a system memory **810**.

[0128] Bus subsystem **802** provides a mechanism for letting the various components and subsystems of computer system **800** communicate with each other as intended. Although bus subsystem **802** is shown schematically as a single bus, alternative embodiments of the bus subsystem may utilize multiple buses. Bus subsystem **802** may be any of several types of bus structures including a memory bus or memory controller, a peripheral bus, and a local bus using any of a variety of bus architectures. For example, such

architectures may include an Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnect (PCI) bus, which can be implemented as a Mezzanine bus manufactured to the IEEE P1386.1 standard.

[0129] Processing unit **804**, which can be implemented as one or more integrated circuits (e.g., a conventional microprocessor or microcontroller), controls the operation of computer system **800**. One or more processors may be included in processing unit **804**. These processors may include single core or multicore processors. In certain embodiments, processing unit **804** may be implemented as one or more independent processing units **832** and/or **834** with single or multicore processors included in each processing unit. In other embodiments, processing unit **804** may also be implemented as a quad-core processing unit formed by integrating two dual-core processors into a single chip.

[0130] In various embodiments, processing unit **804** can execute a variety of programs in response to program code and can maintain multiple concurrently executing programs or processes. At any given time, some, or all of the program code to be executed can be resident in processor(s) **804** and/or in storage subsystem **818**. Through suitable programming, processor(s) **804** can provide various functionalities described above. Computer system **800** may additionally include a processing acceleration unit **806**, which can include a digital signal processor (DSP), a special-purpose processor, and/or the like. In some embodiments, the processing acceleration unit **806** may include or work in conjunction with an acceleration engine such as that disclosed herein to improve computer system functioning.

[0131] I/O subsystem **808** may include user interface input devices and user interface output devices. User interface input devices may include a keyboard, pointing devices such as a mouse or trackball, a touchpad or touch screen incorporated into a display, a scroll wheel, a click wheel, a dial, a button, a switch, a keypad, audio input devices with voice command recognition systems, microphones, and other types of input devices. User interface input devices may include, for example, motion sensing and/or gesture recognition devices such as the Microsoft Kinect® motion sensor that enables users to control and interact with an input device, such as the Microsoft Xbox® 360 game controller, through a natural user interface using gestures and spoken commands. User interface input devices may also include eye gesture recognition devices such as the Google Glass® blink detector that detects eye activity (e.g., 'blinking' while taking pictures and/or making a menu selection) from users and transforms the eye gestures as input into an input device (e.g., Google Glass®). Additionally, user interface input devices may include voice recognition sensing devices that enable users to interact with voice recognition systems (e.g., Siri® navigator), through voice commands.

[0132] User interface input devices may also include, without limitation, three dimensional (3D) mice, joysticks or pointing sticks, gamepads and graphic tablets, and audio/visual devices such as speakers, digital cameras, digital camcorders, portable media players, webcams, image scanners, fingerprint scanners, barcode reader 3D scanners, 3D printers, laser rangefinders, and eye gaze tracking devices. Additionally, user interface input devices may include, for example, medical imaging input devices such as computed tomography, magnetic resonance imaging, position emission

tomography, medical ultrasonography devices. User interface input devices may also include, for example, audio input devices such as MIDI keyboards, digital musical instruments, and the like.

[0133] User interface output devices may include a display subsystem, indicator lights, or non-visual displays such as audio output devices, etc. The display subsystem may be a cathode ray tube (CRT), a flat-panel device, such as that using a liquid crystal display (LCD) or plasma display, a projection device, a touch screen, and the like. In general, use of the term “output device” is intended to include all possible types of devices and mechanisms for outputting information from computer system **800** to a user or other computer. For example, user interface output devices may include, without limitation, a variety of display devices that visually convey text, graphics, and audio/video information such as monitors, printers, speakers, headphones, automotive navigation systems, plotters, voice output devices, and modems.

[0134] Computer system **800** may comprise a storage subsystem **818** that comprises software elements, shown as being currently located within a system memory **810**. System memory **810** may store program instructions that are loadable and executable on processing unit **804**, as well as data generated during the execution of these programs. Depending on the configuration and type of computer system **800**, system memory **810** may be volatile (such as random access memory (RAM)) and/or non-volatile (such as read-only memory (ROM), flash memory, etc.) The RAM typically contains data and/or program modules that are immediately accessible to and/or presently being operated and executed by processing unit **804**. In some implementations, system memory **810** may include multiple different types of memory, such as static random access memory (SRAM) or dynamic random access memory (DRAM). In some implementations, a basic input/output system (BIOS), containing the basic routines that help to transfer information between elements within computer system **800**, such as during start-up, may typically be stored in the ROM. By way of example, and not limitation, system memory **810** also illustrates application programs **812**, which may include client applications, Web browsers, mid-tier applications, relational database management systems (RDBMS), etc., program data **814**, and an operating system **816**. By way of example, operating system **816** may include various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems, a variety of commercially-available UNIX® or UNIX-like operating systems (including without limitation the variety of GNU/Linux operating systems, the Google Chrome® OS, and the like) and/or mobile operating systems such as iOS, Windows® Phone, Android® OS, BlackBerry® 10 OS, and Palm® OS operating systems.

[0135] Storage subsystem **818** may also provide a tangible computer-readable storage medium for storing the basic programming and data constructs that provide the functionality of some embodiments. Software (programs, code modules, instructions) that when executed by a processor provide the functionality described above may be stored in storage subsystem **818**. These software modules or instructions may be executed by processing unit **804**. Storage subsystem **818** may also provide a repository for storing data used in accordance with the present invention.

[0136] Storage subsystem **800** may also include a computer-readable storage media reader **820** that can further be connected to computer-readable storage media **822**. Together and optionally, in combination with system memory **810**, computer-readable storage media **822** may comprehensively represent remote, local, fixed, and/or removable storage devices plus storage media for temporarily and/or more permanently containing, storing, transmitting, and retrieving computer-readable information.

[0137] Computer-readable storage media **822** containing code, or portions of code, can also include any appropriate media known or used in the art, including storage media and communication media, such as but not limited to, volatile and non-volatile, removable, and non-removable media implemented in any method or technology for storage and/or transmission of information. This can include tangible computer-readable storage media such as RAM, ROM, electronically erasable programmable ROM (EEPROM), flash memory or other memory technology, CD-ROM, digital versatile disk (DVD), or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or other tangible computer readable media. This can also include nontangible computer-readable media, such as data signals, data transmissions, or any other medium which can be used to transmit the desired information, and which can be accessed by computing system **800**.

[0138] By way of example, computer-readable storage media **822** may include a hard disk drive that reads from or writes to non-removable, nonvolatile magnetic media, a magnetic disk drive that reads from or writes to a removable, nonvolatile magnetic disk, and an optical disk drive that reads from or writes to a removable, nonvolatile optical disk such as a CD ROM, DVD, and Blu-Ray® disk, or other optical media. Computer-readable storage media **822** may include, but is not limited to, Zip® drives, flash memory cards, universal serial bus (USB) flash drives, secure digital (SD) cards, DVD disks, digital video tape, and the like. Computer-readable storage media **822** may also include, solid-state drives (SSD) based on non-volatile memory such as flash-memory based SSDs, enterprise flash drives, solid state ROM, and the like, SSDs based on volatile memory such as solid state RAM, dynamic RAM, static RAM, DRAM-based SSDs, magneto resistive RAM (MRAM) SSDs, and hybrid SSDs that use a combination of DRAM and flash memory based SSDs. The disk drives and their associated computer-readable media may provide non-volatile storage of computer-readable instructions, data structures, program modules, and other data for computer system **800**.

[0139] Communications subsystem **824** provides an interface to other computer systems and networks. Communications subsystem **824** serves as an interface for receiving data from and transmitting data to other systems from computer system **800**. For example, communications subsystem **824** may enable computer system **800** to connect to one or more devices via the Internet. In some embodiments communications subsystem **824** can include radio frequency (RF) transceiver components for accessing wireless voice and/or data networks (e.g., using cellular telephone technology, advanced data network technology, such as 5G, 4G or EDGE (enhanced data rates for global evolution), Wi-Fi (IEEE 802.11 family standards, or other mobile communication technologies, or any combination thereof), global position-

ing system (GPS) receiver components, and/or other components. In some embodiments communications subsystem **824** can provide wired network connectivity (e.g., Ethernet) in addition to or instead of a wireless interface.

[0140] In some embodiments, communications subsystem **824** may also receive input communication in the form of structured and/or unstructured data feeds **826**, event streams **828**, event updates **830**, and the like on behalf of one or more users who may use computer system **800**. By way of example, communications subsystem **824** may be configured to receive data feeds **826** in real-time from users of social networks and/or other communication services such as Twitter® feeds, Facebook® updates, web feeds such as Rich Site Summary (RSS) feeds, and/or real-time updates from one or more third party information sources.

[0141] Additionally, communications subsystem **824** may also be configured to receive data in the form of continuous data streams, which may include event streams **828** of real-time events and/or event updates **830**, that may be continuous or unbounded in nature with no explicit end. Examples of applications that generate continuous data may include, for example, sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), click-stream analysis tools, automobile traffic monitoring, and the like. Communications subsystem **824** may also be configured to output the structured and/or unstructured data feeds **826**, event streams **828**, event updates **830**, and the like to one or more databases that may be in communication with one or more streaming data source computers coupled to computer system **800**.

[0142] Computer system **800** can be one of various types, including a handheld portable device (e.g., an iPhone® cellular phone, an iPad® computing tablet, a PDA), a wearable device (e.g., a Google Glass® head mounted display), a PC, a workstation, a mainframe, a kiosk, a server rack, or any other data processing system. Due to the ever-changing nature of computers and networks, the description of computer system **800** depicted in the figure is intended only as a specific example. Many other configurations having more or fewer components than the system depicted in the figure are possible. For example, customized hardware might also be used and/or particular elements might be implemented in hardware, firmware, software (including applets), or a combination. Further, connection to other computing devices, such as network input/output devices, may be employed. Based on the disclosure and teachings provided herein, a person of ordinary skill in the art will appreciate other ways and/or methods to implement the various embodiments.

[0143] In the foregoing description, for the purposes of explanation, numerous specific details were set forth in order to provide a thorough understanding of various embodiments of the present invention. It will be apparent, however, to one skilled in the art that embodiments of the present invention may be practiced without some of these specific details. In other instances, well-known structures and devices are shown in block diagram form.

[0144] The foregoing description provides exemplary embodiments only, and is not intended to limit the scope, applicability, or configuration of the disclosure. Rather, the foregoing description of the exemplary embodiments will provide those skilled in the art with an enabling description for implementing an exemplary embodiment. It should be

understood that various changes may be made in the function and arrangement of elements without departing from the spirit and scope of the invention as set forth in the appended claims.

[0145] Specific details are given in the foregoing description to provide a thorough understanding of the embodiments. However, it will be understood by one of ordinary skill in the art that the embodiments may be practiced without these specific details. For example, circuits, systems, networks, processes, and other components may have been shown as components in block diagram form in order not to obscure the embodiments in unnecessary detail. In other instances, well-known circuits, processes, algorithms, structures, and techniques may have been shown without unnecessary detail in order to avoid obscuring the embodiments.

[0146] Also, it is noted that individual embodiments may have been described as a process which is depicted as a flowchart, a flow diagram, a data flow diagram, a structure diagram, or a block diagram. Although a flowchart may have described the operations as a sequential process, many of the operations can be performed in parallel or concurrently. In addition, the order of the operations may be re-arranged. A process is terminated when its operations are completed but could have additional steps not included in a figure. A process may correspond to a method, a function, a procedure, a subroutine, a subprogram, etc. When a process corresponds to a function, its termination can correspond to a return of the function to the calling function or the main function.

[0147] The term “computer-readable medium” includes but is not limited to portable or fixed storage devices, optical storage devices, wireless channels, and various other mediums capable of storing, containing, or carrying instruction(s) and/or data. A code segment or machine-executable instructions may represent a procedure, a function, a subprogram, a program, a routine, a subroutine, a module, a software package, a class, or any combination of instructions, data structures, or program statements. A code segment may be coupled to another code segment or a hardware circuit by passing and/or receiving information, data, arguments, parameters, or memory contents. Information, arguments, parameters, data, etc., may be passed, forwarded, or transmitted via any suitable means including memory sharing, message passing, token passing, network transmission, etc.

[0148] Furthermore, embodiments may be implemented by hardware, software, firmware, middleware, microcode, hardware description languages, or any combination thereof. When implemented in software, firmware, middleware or microcode, the program code or code segments to perform the necessary tasks may be stored in a machine readable medium. A processor(s) may perform the necessary tasks.

[0149] In the foregoing specification, aspects of the invention are described with reference to specific embodiments thereof, but those skilled in the art will recognize that the invention is not limited thereto. Various features and aspects of the above-described invention may be used individually or jointly. Further, embodiments can be utilized in any number of environments and applications beyond those described herein without departing from the broader spirit and scope of the specification. The specification and drawings are, accordingly, to be regarded as illustrative rather than restrictive.

[0150] Additionally, for the purposes of illustration, methods were described in a particular order. It should be appreciated that in alternate embodiments, the methods may be performed in a different order than that described. It should also be appreciated that the methods described above may be performed by hardware components or may be embodied in sequences of machine-executable instructions, which may be used to cause a machine, such as a general-purpose or special-purpose processor or logic circuits programmed with the instructions to perform the methods. These machine-executable instructions may be stored on one or more machine readable mediums, such as CD-ROMs or other type of optical disks, floppy diskettes, ROMs, RAMS, EPROMs, EEPROMs, magnetic or optical cards, flash memory, or other types of machine-readable mediums suitable for storing electronic instructions. Alternatively, the methods may be performed by a combination of hardware and software.

[0151] Also, the terms in the claims have their plain, ordinary meaning unless otherwise explicitly and clearly defined by the patentee. The indefinite articles “a” or “an,” as used in the claims, are defined herein to mean one or more than one of the element that the particular article introduces; and subsequent use of the definite article “the” is not intended to negate that meaning. Furthermore, the use of ordinal number terms, such as “first,” “second,” etc., to clarify different elements in the claims is not intended to impart a particular position in a series, or any other sequential character or order, to the elements to which the ordinal number terms have been applied.

What is claimed:

1. A computer-implemented method comprising:
receiving, at a computing system, a file system command that targets a clone directory located within a special directory of a file system; and
in response to the command:
creating a snapshot of a target file system;
cloning the snapshot into the clone directory to generate a live, read-write file system instance;
mapping an address of the target file system into the clone directory; and
linking the clone directory to a head node, wherein the head node presents the clone directory as visible to clients through a unified mount point.
2. The computer implemented method of claim 1, further comprising mirroring security privileges and administrative access from the target file system to the clone directory.
3. The computer implemented method of claim 1, wherein files in the clone directory are read-write files.
4. The computer implemented method of claim 1, further comprising receiving a selection of the snapshot of the target file system that is stored in a snapshot directory.
5. The computer implemented method of claim 1, wherein the snapshot is a point-in-time reference of the target file system.
6. The computer implemented method of claim 1, further comprising copying file system properties of the target file system, wherein the file system properties include one or more of compartment, tags, display name, keys, and mount target export information.
7. The computer implemented method of claim 1, further comprising copying linking metadata from the target file system to the clone directory.

8. A computer-program product tangibly embodied in a non-transitory machine-readable storage medium, including instructions configured to cause one or more data processors to perform a set of operations comprising:

- receiving, at a computing system, a file system command that targets a clone directory located within a special directory of a file system; and
- in response to the command:
creating a snapshot of a target file system;
cloning the snapshot into the clone directory to generate a live, read-write file system instance;
mapping an address of the target file system into the clone directory; and
linking the clone directory to a head node, wherein the head node presents the clone directory as visible to clients through a unified mount point.

9. The computer-program product of claim 8, wherein the set of operations further comprises mirroring security privileges and administrative access from the target file system to the clone directory.

10. The computer-program product of claim 8, wherein files in the clone directory are read-write files.

11. The computer-program product of claim 8, wherein the set of operations further comprises receiving a selection of the snapshot of the target file system that is stored in a snapshot directory.

12. The computer-program product of claim 8, wherein the snapshot is a point-in-time reference of the target file system.

13. The computer-program product of claim 8, wherein the set of operations further comprises copying file system properties of the target file system, wherein the file system properties include one or more of compartment, tags, display name, keys, and mount target export information.

14. The computer-program product of claim 8, wherein the set of operations further comprises copying linking metadata from the target file system to the clone directory.

15. A system comprising:
one or more processors;
one or more non-transitory computer-readable media storing instructions, which, when executed by the system, cause the system to perform a set of operations comprising:
receiving, at a computing system, a file system command that targets a clone directory located within a special directory of a file system; and
in response to the command:
creating a snapshot of a target file system;
cloning the snapshot into the clone directory to generate a live, read-write file system instance;
mapping an address of the target file system into the clone directory; and
linking the clone directory to a head node, wherein the head node presents the clone directory as visible to clients through a unified mount point.

16. The system of claim 15, wherein the set of operations further comprises mirroring security privileges and administrative access from the target file system to the clone directory.

17. The system of claim 15, wherein files in the clone directory are read-write files.

18. The system of claim 15, wherein the set of operations further comprises receiving a selection of the snapshot of the target file system that is stored in a snapshot directory.

19. The system of claim **15**, wherein the snapshot is a point-in-time reference of the target file system.

20. The system of claim **15**, wherein the set of operations further comprises copying file system properties of the target file system, wherein the file system properties include one or more of compartment, tags, display name, keys, and mount target export information.

* * * * *